Dokumentasi Resmi & Sintaks Ruby (Ruby 3.2/3.3 Up-to-Date)

Bahasa: Ruby | Total Chunk/Dokumen: 124 | Disiapkan untuk RAG System

[ruby_001] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Overview

Sumber: ht

= Literals
Literals create objects you can use in your program. Literals include:
* {Boolean and Nil Literals}[#label-BooleanandNilLiterals]
* {Numeric Literals}[#label-NumericLiterals]
* {Integer Literals}[#label-IntegerLiterals]
* {Float Literals}[#label-FloatLiterals]
* {Rational Literals}[#label-RationalLiterals]
* {Complex Literals}[#label-ComplexLiterals]
* {String Literals}[#label-StringLiterals]
* {Here Document Literals}[#label-HereDocumentLiterals]
* {Symbol Literals}[#label-SymbolLiterals]
* {Array Literals}[#label-ArrayLiterals]
* {Hash Literals}[#label-HashLiterals]
* {Range Literals}[#label-RangeLiterals]
* {Regexp Literals}[#label-RegexpLiterals]
* {Lambda Proc Literals}[#label-LambdaProcLiterals]
* {Percent Literals}[#label-PercentLiterals]
* {%q: Non-Interpolable String Literals}[#label-25q-3ANon-InterpolableStringLiterals]
* {% and %Q: Interpolable String Literals}[#label-25and-25Q-3AInterpolableStringLiterals]
* {%w and %W: String-Array Literals}[#label-25wand-25W-3AString-ArrayLiterals]
* {%i and %I: Symbol-Array Literals}[#label-25iand-25I-3ASymbol-ArrayLiterals]
* {%r: Regexp Literals}[#label-25r-3ARegexpLiterals]
* {%s: Symbol Literals}[#label-25s-3ASymbolLiterals]
* {%x: Backtick Literals}[#label-25x-3ABacktick+Literals]

[ruby_002] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Boolean and Nil Literals

Sumber: ht

== Boolean and Nil Literals
nil and false are both false values. nil is sometimes used to indicate
"no value" or "unknown" but evaluates to false in conditional expressions.
true is a true value. All objects except nil and false evaluate to a
true value in conditional expressions.

[ruby_003] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Integer Literals

Sumber: ht

=== \Integer Literals
You can write integers of any size as follows:
1234
1_234
These numbers have the same value, 1,234. The underscore may be used to enhance readability for humans. You may place an underscore anywhere in the number.
You can use a special prefix to write numbers in decimal, hexadecimal, octal or binary formats. For decimal numbers use a prefix of 0d, for hexadecimal numbers use a prefix of 0x, for octal numbers use a prefix of 0 or 0o, for binary numbers use a prefix of 0b. The alphabetic component of the number is not case-sensitive.
Examples:
0d170
0D170
0xaa
0xAa
0xAA
0Xaa
0XAa
0XaA
0252
0o252
0O252
0b10101010
0B10101010
All these numbers have the same decimal value, 170. Like integers and floats you may use an underscore for readability.

[ruby_004] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Float Literals

Sumber: ht

=== \Float Literals
Floating-point numbers may be written as follows:
12.34
1234e-2
1.234E1
These numbers have the same value, 12.34. You may use underscores in floating point numbers as well.

[ruby_005] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Rational Literals

Sumber: ht

=== \Rational Literals
You can write a Rational literal using a special suffix, 'r'.
Examples:
1r   # => (1/1)
2/3r   # => (2/3)   # With denominator.
-1r   # => (-1/1) # With signs.
-2/3r   # => (-2/3)
2/-3r   # => (-2/3)
-2/-3r   # => (2/3)
1/3r   # => (1/3)
1.2r   # => (6/5)   # With fractional part.
1_1/2_1r # => (11/21) # With embedded underscores.
2/4r   # => (1/2)   # Automatically reduced.
Syntax:
= ['/'] 'r'
= [ ] [ ]
= '.'
= [sign]
= '-' | '+'
= { | '_' }
= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
Note this, which is parsed as \Float numerator 1.2
divided by \Rational denominator 3r,
resulting in a \Float:
1.2/3r # => 0.39999999999999997

[ruby_006] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Complex Literals

Sumber: ht

=== \Complex Literals
You can write a Complex number as follows (suffixed i):
1i   #=> (01i)
1i * 1i   #=> (-10i)
Also \Rational numbers may be imaginary numbers.
12.3ri   #=> (0(123/10)*i)
i must be placed after r+; the opposite is not allowed.
12.3ir   #=> Syntax error

[ruby_007] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Double-Quoted \String Literals

Sumber: ht

=== Double-Quoted \String Literals
The most common way of writing strings is using ":
"This is a string."
The string may be many lines long.
Any internal " must be escaped:
"This string has a quote: \". As you can see, it is escaped"
Double-quoted strings allow escape sequences described in
{Escape Sequences}[#label-EscapeSequences].
In a double-quoted string,
any other character following a backslash is interpreted as the
character itself.
Double-quoted strings allow interpolation of other values using
#{...}:
"One plus one is two: #{1 1}"
Any expression may be placed inside the interpolated section, but it's best to
keep the expression small for readability.
You can also use #@foo, #@@foo and #$foo as a
shorthand for, respectively, #{ @foo }, #{ @@foo } and
#{ $foo }.
See also:
* {% and %Q: Interpolable String Literals}[#label-25and-25Q-3AInterpolableString+Literals]

[ruby_008] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Single-Quoted \String Literals

Sumber: ht

=== Single-Quoted \String Literals
Interpolation may be disabled by escaping the "#" character or using
single-quoted strings:
'#{1 1}' #=> "\#{1 1}"
In addition to disabling interpolation, single-quoted strings also disable all
escape sequences except for the single-quote (\') and backslash
(\\\\).
In a single-quoted string,
any other character following a backslash is interpreted as is:
a backslash and the character itself.
See also:
* {%q: Non-Interpolable String Literals}[#label-25q-3ANon-InterpolableString+Literals]

[ruby_009] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Literal String Concatenation

Sumber: ht

=== Literal String Concatenation
Adjacent string literals are automatically concatenated by the interpreter:
"con" "cat" "en" "at" "ion" #=> "concatenation"
"This string contains "\
"no newlines." #=> "This string contains no newlines."
Any combination of adjacent single-quote, double-quote, percent strings will
be concatenated as long as a percent-string is not last.
%q{a} 'b' "c" #=> "abc"
"a" 'b' %q{c} #=> NoMethodError: undefined method 'q' for main

[ruby_010] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Character Literal

Sumber: ht

=== Character Literal
There is also a character literal notation to represent single
character strings, which syntax is a question mark (?)
followed by a single character or escape sequence (except continuation line)
that corresponds to a single codepoint in the script encoding:
?a #=> "a"
?abc #=> SyntaxError
?\n #=> "\n"
?\s #=> " "
?\\ #=> "\\"
?\u{41} #=> "A"
?\C-a #=> "\x01"
?\M-a #=> "\xE1"
?\M-\C-a #=> "\x81"
?\C-\M-a #=> "\x81", same as above
?? #=> "?"

[ruby_011] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Escape Sequences

Sumber: ht

=== Escape Sequences
Some characters can be represented as escape sequences in
double-quoted strings,
character literals,
here document literals (non-quoted, double-quoted, and with backticks),
double-quoted symbols,
double-quoted symbol keys in Hash literals,
Regexp literals, and
several percent literals (%, %Q, %W, %I, %r, %x).
They allow escape sequences such as \n for
newline, \t for tab, etc. The full list of supported escape

sequences are as follows:
\a bell, ASCII 07h (BEL)
\b backspace, ASCII 08h (BS)
\t horizontal tab, ASCII 09h (TAB)
\n newline (line feed), ASCII 0Ah (LF)
\v vertical tab, ASCII 0Bh (VT)
\f form feed, ASCII 0Ch (FF)
\r carriage return, ASCII 0Dh (CR)
\e escape, ASCII 1Bh (ESC)
\s space, ASCII 20h (SPC)
\\ backslash, \
\nnn octal bit pattern, where nnn is 1-3 octal digits ([0-7])
\xnn hexadecimal bit pattern, where nn is 1-2 hexadecimal digits ([0-9a-fA-F])
\unnnn Unicode character, where nnnn is exactly 4 hexadecimal digits ([0-9a-fA-F])
\u{nnnn ...} Unicode character(s), where each nnnn is 1-6 hexadecimal digits ([0-9a-fA-F])
\cx or \C-x control character, where x is an ASCII printable character
\M-x meta character, where x is an ASCII printable character
\M-\C-x meta control character, where x is an ASCII printable character
\M-\cx same as above
\c\M-x same as above
\c? or \C-? delete, ASCII 7Fh (DEL)
\ continuation line (empty string)
The last one, \, represents an empty string instead of a character.
It is used to fold a line in a string.

[ruby_012] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Here Document Literals

Sumber: ht

=== Here Document Literals
If you are writing a large block of text you may use a "here document" or
"heredoc":
expected_result = <<EOF and ends with the
next line that starts with EOF. The result includes the ending
newline.
You may use any identifier with a heredoc, but all-uppercase identifiers are
typically used.
You may indent the ending identifier if you place a "-" after <<
expected_result = <<-EOF
expected_result = <<~SQUIGGLY_HEREDOC
This would contain specially formatted text.
That might span many lines
SQUIGGLY_HEREDOC
The indentation of the least-indented line will be removed from each line of
the content. Note that empty lines and lines consisting solely of literal tabs
and spaces will be ignored for the purposes of determining indentation, but
escaped tabs and spaces are considered non-indentation characters.
For the purpose of measuring an indentation, a horizontal tab is regarded as a
sequence of one to eight spaces such that the column position corresponding to
its end is a multiple of eight. The amount to be removed is counted in terms
of the number of spaces. If the boundary appears in the middle of a tab, that

tab is not removed.
A heredoc allows interpolation and the escape sequences described in
{Escape Sequences}[#label-EscapeSequences].
You may disable interpolation and the escaping by surrounding the opening
identifier with single quotes:
expected_result = <<-'EXPECTED'
One plus one is #{1 1}
EXPECTED
p expected_result # prints: "One plus one is \#{1 1}\n"
The identifier may also be surrounded with double quotes (which is the same as
no quotes) or with backticks. When surrounded by backticks the HEREDOC
behaves like Kernel#`:
puts <<-`HEREDOC`
cat #{__FILE__}
HEREDOC
When surrounding with quotes, any character but that quote and newline
(CR and/

Contoh Kode / Perintah CLI:

```
expected_result = <<~SQUIGGLY_HEREDOC
This would contain specially formatted text.
That might span many lines
SQUIGGLY_HEREDOC
```

[ruby_013] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Symbol Literals

Sumber: ht

== \Symbol Literals
A Symbol represents a name inside the ruby interpreter. See Symbol for more
details on what symbols are and when ruby creates them internally.
You may reference a symbol using a colon: :my_symbol.
You may also create symbols by interpolation and escape sequences described in
{Escape Sequences}[#label-EscapeSequences] with double-quotes:
:"my_symbol1"
:"my_symbol#{1 1}"
:"foo\sbar"
Like strings, a single-quote may be used to disable interpolation and
escape sequences:
:'my_symbol#{1 1}' #=> :"my_symbol\#{1 1}"
When creating a Hash, there is a special syntax for referencing a Symbol as
well.
See also:
* {%s: Symbol Literals}[#label-25s-3ASymbolLiterals]

[ruby_014] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Array Literals

Sumber: ht

== \Array Literals
An array is created using the objects between [and]:
[1, 2, 3]
You may place expressions inside the array:
[1, 1 1, 1 2]
[1, [1 1, [1 2]]]
See also:
* {%w and %W: String-Array Literals}[#label-25wand-25W-3AString-ArrayLiterals]
* {%i and %I: Symbol-Array Literals}[#label-25iand-25I-3ASymbol-ArrayLiterals]
See Array for the methods you may use with an array.

[ruby_015] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Hash Literals

Sumber: ht

== \Hash Literals
A hash is created using key-value pairs between { and }:
{ "a" => 1, "b" => 2 }
Both the key and value may be any object.
You can create a hash using symbol keys with the following syntax:
{ a: 1, b: 2 }
This same syntax is used for keyword arguments for a method.
Like Symbol literals, you can quote symbol keys.
{ "a 1": 1, "b #{1 + 1}": 2 }
is equal to
{ :"a 1" => 1, :"b 2" => 2 }
Hash values can be omitted, meaning that value will be fetched from the context
by the name of the key:
x = 100
y = 200
h = { x:, y: }
#=> {:x=>100, :y=>200}
See Hash for the methods you may use with a hash.

[ruby_016] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Range Literals

Sumber: ht

== \Range Literals
A range represents an interval of values. The range may include or exclude
its ending value.
(1..2) # includes its ending value
(1...2) # excludes its ending value
(1..)   # endless range, representing infinite sequence from 1 to Infinity
(..1)   # beginless range, representing infinite sequence from -Infinity to 1
You may create a range of any object. See the Range documentation for details
on the methods you need to implement.

[ruby_017] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): \Regexp Literals

Sumber: ht

== \Regexp Literals
A regular expression may be created using leading and trailing
slash ('/') characters:
re = /foo/ # => /foo/
re.class # => Regexp
The trailing slash may be followed by one or more modifiers characters
that set modes for the regexp.
See {Regexp modes}[rdoc-ref:Regexp@Modes] for details.
Interpolation may be used inside regular expressions along with escaped
characters. Note that a regular expression may require additional escaped
characters than a string.
See also:
* {%r: Regexp Literals}[#label-25r-3ARegexpLiterals]
See Regexp for a description of the syntax of regular expressions.

[ruby_018] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Lambda Proc Literals

Sumber: ht

== Lambda Proc Literals
A lambda proc can be created with ->:
-> { 1 1 }
Calling the above proc will give a result of 2.
You can require arguments for the proc as follows:
->(v) { 1 v }
This proc will add one to its argument.

[ruby_019] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): Percent Literals

Sumber: ht

== Percent Literals
Each of the literals in described in this section
may use these paired delimiters:
* [and].
* (and).
* { and }.
* and >.
* Non-alphanumeric ASCII character except above, as both beginning and ending delimiters.
The delimiters can be escaped with a backslash.
However, the first four pairs (brackets, parenthesis, braces, and
angle brackets) are allowed without backslash as far as they are correctly
paired.
These are demonstrated in the next section.

[ruby_020] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>%q</tt>: Non-Interpolable String Literals

Sumber: ht

=== %q: Non-Interpolable String Literals
You can write a non-interpolable string with %q.
The created string is the same as if you created it with single quotes:
%q[foo bar baz] # => "foo bar baz" # Using [].
%q(foo bar baz) # => "foo bar baz" # Using ().
%q{foo bar baz} # => "foo bar baz" # Using {}.
%q # => "foo bar baz" # Using <>.
%q|foo bar baz| # => "foo bar baz" # Using two |.
%q:foo bar baz: # => "foo bar baz" # Using two :.
%q(1 1 is #{1 1}) # => "1 1 is \#{1 1}" # No interpolation.
%q[foo[bar]baz] # => "foo[bar]baz" # brackets can be nested.
%q(foo(bar)baz) # => "foo(bar)baz" # parenthesis can be nested.
%q{foo{bar}baz} # => "foo{bar}baz" # braces can be nested.
%qbaz> # => "foobaz" # angle brackets can be nested.
This is similar to single-quoted string but only backslashes and
the specified delimiters can be escaped with a backslash.

[ruby_021] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>% and %Q</tt>: Interpolable String Literals

Sumber: ht

=== % and %Q: Interpolable String Literals
You can write an interpolable string with %Q
or with its alias %:
%[foo bar baz] # => "foo bar baz"
%(1 1 is #{1 1}) # => "1 1 is 2" # Interpolation.
This is similar to double-quoted string.
It allow escape sequences described in
{Escape Sequences}[#label-EscapeSequences].
Other escaped characters (a backslash followed by a character) are
interpreted as the character.

[ruby_022] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>%w and %W</tt>: String-Array Literals

Sumber: ht

=== %w and %W: String-Array Literals
You can write an array of strings as whitespace-separated words
with %w (non-interpolable) or %W (interpolable):
%w[foo bar baz] # => ["foo", "bar", "baz"]
%w[1 % *] # => ["1", "%", "*"]
Use backslash to embed spaces in the strings.
%w[foo\ bar baz\ bat] # => ["foo bar", "baz bat"]
%W[foo\ bar baz\ bat] # => ["foo bar", "baz bat"]
%w(#{1 1}) # => ["\#{1", "", "1}"]
%W(#{1 1}) # => ["2"]

```
# The nested delimiters evaluated to a flat array of strings
# (not nested array).
%w[foo[bar baz]qux]   # => ["foo[bar", "baz]qux"]
The interpolated string is treated as a single word even if it contains
whitespace.
s = "bar baz"
%W[foo #{s} zot] #=> ["foo", "bar baz", "zot"]
%W[foo #{"bar baz  zot"} qux] # => ["foo", "bar baz  zot", "qux"]
The following characters are considered as white spaces to separate words:
* space, ASCII 20h (SPC)
* form feed, ASCII 0Ch (FF)
* newline (line feed), ASCII 0Ah (LF)
* carriage return, ASCII 0Dh (CR)
* horizontal tab, ASCII 09h (TAB)
* vertical tab, ASCII 0Bh (VT)
The white space characters can be escaped with a backslash to make them
part of a word.
%W allow escape sequences described in
{Escape Sequences}[#label-EscapeSequences].
However the continuation line \ is not usable because
it is interpreted as the escaped newline described above.
```

[ruby_023] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>%i and %I</tt>: Symbol-Array Literals

Sumber: ht

```
=== %i and %I: Symbol-Array Literals
You can write an array of symbols as whitespace-separated words
with %i (non-interpolable) or %I (interpolable):
%i[foo bar baz]       # => [:foo, :bar, :baz]
%i[1 % *]             # => [:"1", :%, :*]
# Use backslash to embed spaces in the symbols.
%i[foo\ bar baz\ bat] # => [:"foo bar", :"baz bat"]
%I[foo\ bar baz\ bat] # => [:"foo bar", :"baz bat"]
%i(#{1  1})          # => [:"\#{1", :, :"1}"]
%I(#{1  1})          # => [:"2"]
The white space characters and its escapes are interpreted as the same as
string-array literals described in
{%w and %W: String-Array Literals}[#label-25wand-25W-3AString-Array+Literals].
```

[ruby_024] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>%s</tt>: Symbol Literals

Sumber: ht

```
=== %s: Symbol Literals
You can write a symbol with %s:
%s[foo]     # => :foo
%s[foo bar] # => :"foo bar"
This is non-interpolable.
No interpolation allowed.
```

Only backslashes and the specified delimiters can be escaped with a backslash.

[ruby_025] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>%r</tt>: Regexp Literals

Sumber: ht

=== %r: Regexp Literals
You can write a regular expression with %r;
the character used as the leading and trailing delimiter
may be (almost) any character:
%r/foo/ # => /foo/
%r:name/value pair: # => /name\/value pair/
A few "symmetrical" character pairs may be used as delimiters:
%r[foo] # => /foo/
%r{foo} # => /foo/
%r(foo) # => /foo/
%r # => /foo/
The trailing delimiter may be followed by one or more modifier characters
that set modes for the regexp.
See {Regexp modes}[rdoc-ref:Regexp@Modes] for details.

[ruby_026] Ruby - Literals & Basic Types

Literals (Numbers, Strings, Arrays, Hashes, Symbols, Ranges, Regexp): <tt>%x</tt>: Backtick Literals

Sumber: ht

=== %x: Backtick Literals
You can write and execute a shell command with %x:
%x(echo 1) # => "1\n"
%x[echo #{1 2}] # => "3\n"
%x[echo \u0030] # => "0\n"
This is interpolable.
%x allow escape sequences described in
{Escape Sequences}[#label-EscapeSequences].

[ruby_027] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Overview

Sumber: ht

= Assignment
In Ruby, assignment uses the = (equals sign) character. This
example assigns the number five to the local variable v:
v = 5
Assignment creates a local variable if the variable was not previously
referenced.
An assignment expression result is always the assigned value, including
{assignment methods}[rdoc-ref:@Assignment+Methods].

[ruby_028] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Local Variable Names

Sumber: ht

== Local Variable Names
A local variable name must start with a lowercase US-ASCII letter or a
character with the eight bit set. Typically local variables are US-ASCII
compatible since the keys to type them exist on all keyboards.
(Ruby programs must be written in a US-ASCII-compatible character set. In
such character sets if the eight bit is set it indicates an extended
character. Ruby allows local variables to contain such characters.)
A local variable name may contain letters, numbers, an _
(underscore or low line) or a character with the eighth bit set.

[ruby_029] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Local Variable Scope

Sumber: ht

== Local Variable Scope
Once a local variable name has been assigned-to all uses of the name for the
rest of the scope are considered local variables.
Here is an example:
1.times do
a = 1
puts "local variables in the block: #{local_variables.join ", "}"
end
puts "no local variables outside the block" if local_variables.empty?
This prints:
local variables in the block: a
no local variables outside the block
Since the block creates a new scope, any local variables created inside it do
not leak to the surrounding scope.
Variables defined in an outer scope appear inner scope:
a = 0
1.times do
puts "local variables: #{local_variables.join ", "}"
end
This prints:
local variables: a
You may isolate variables in a block from the outer scope by listing them
following a ; in the block's arguments. See the documentation
for block local variables in the {calling
methods}[rdoc-ref:syntax/calling_methods.rdoc] documentation for an example.
See also Kernel#local_variables, but note that a for loop does not create a
new scope like a block does.

[ruby_030] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Local Variables and Methods

Sumber: ht

== Local Variables and Methods
In Ruby local variable names and method names are nearly identical. If you
have not assigned to one of these ambiguous names ruby will assume you wish to
call a method. Once you have assigned to the name ruby will assume you wish
to reference a local variable.
The local variable is created when the parser encounters the assignment, not
when the assignment occurs:
a = 0 if false # does not assign to a
p local_variables # prints [:a]
p a # prints nil
The similarity between method and local variable names can lead to confusing
code, for example:
def big_calculation
42 # pretend this takes a long time
end
big_calculation = big_calculation()
Now any reference to big_calculation is considered a local variable and will
be cached. To call the method, use self.big_calculation.
You can force a method call by using empty argument parentheses as shown above
or by using an explicit receiver like self. Using an explicit
receiver may raise a NameError if the method's visibility is not public or the
receiver is the literal self.
Another commonly confusing case is when using a modifier if:
p a if a = 0.zero?
Rather than printing "true" you receive a NameError, "undefined local variable
or method `a'". Since ruby parses the bare a left of the if first and has
not yet seen an assignment to a it assumes you wish to call a method. Ruby
then sees the assignment to a and will assume you are referencing a local
variable.
The confusion comes from the out-of-order execution of the expression. First
the local variable is assigned-to then you attempt to call a nonexistent
method.

[ruby_031] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Local Variables and eval

Sumber: ht

== Local Variables and eval
Using eval to evaluate Ruby code will allow access to local variables defined
in the same scope, even if the local variables are not defined until after the
call to eval. However, local variables defined inside the call to eval
will not be reflected in the surrounding scope. Inside the call to eval,
local variables defined in the surrounding scope and local variables defined
inside the call to eval will be accessible. However, you will not be able
to access local variables defined in previous or subsequent calls to eval in
the same scope. Consider each eval call a separate nested scope. Example:

```
def m
eval "bar = 1"
lvs = eval "baz = 2; ary = [local_variables, foo, baz]; x = 2; ary"
eval "quux = 3"
foo = 1
lvs  [[:baz, :ary, :x, :lvs, :foo], nil, 2, [:lvs, :foo]]
```

[ruby_032] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Instance Variables

Sumber: ht

== Instance Variables
Instance variables are shared across all methods for the same object.
An instance variable must start with a @ ("at" sign or
commercial at). Otherwise instance variable names follow the rules as local
variable names. Since the instance variable starts with an @ the
second character may be an upper-case letter.
Here is an example of instance variable usage:
class C
def initialize(value)
@instance_variable = value
end
def value
@instance_variable
end
end
object1 = C.new "some value"
object2 = C.new "other value"
p object1.value # prints "some value"
p object2.value # prints "other value"
An uninitialized instance variable has a value of nil.
The value method has access to the value set by the initialize method, but
only for the same object.

Contoh Kode / Perintah CLI:

```
def value
@instance_variable
end
```

[ruby_033] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Class Variables

Sumber: ht

== Class Variables
Class variables are shared between a class, its subclasses and its instances.
A class variable must start with a @@ (two "at" signs). The rest
of the name follows the same rules as instance variables.
Here is an example:
class A
@@class_variable = 0
def value

@@class_variable
end
def update
@@class_variable = @@class_variable 1
end
end
class B < A
def update
@@class_variable = @@class_variable 2
end
end
a = A.new
b = B.new
puts "A value: #{a.value}"
puts "B value: #{b.value}"
This prints:
A value: 0
B value: 0
Continuing with the same example, we can update using objects from either class and the value is shared:
puts "update A"
a.update
puts "A value: #{a.value}"
puts "B value: #{b.value}"
puts "update B"
b.update
puts "A value: #{a.value}"
puts "B value: #{b.value}"
puts "update A"
a.update
puts "A value: #{a.value}"
puts "B value: #{b.value}"
This prints:
update A
A value: 1
B value: 1
update B
A value: 3
B value: 3
update A
A value: 4
B value: 4
Accessing an uninitialized class variable will raise a NameError exception.
Note that classes have instance variables because classes are objects, so try not to confuse class and instance variables.

Contoh Kode / Perintah CLI:

```
def value
@@class_variable
end
def update
@@class_variable = @@class_variable + 1
end
```

[ruby_034] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Global Variables

Sumber: ht

== Global Variables
Global variables are accessible everywhere.
Global variables start with a $ (dollar sign). The rest of the
name follows the same rules as instance variables.
Here is an example:
$global = 0
class C
puts "in a class: #{$global}"
def my_method
puts "in a method: #{$global}"
$global = $global 1
$other_global = 3
end
end
C.new.my_method
puts "at top-level, $global: #{$global}, $other_global: #{$other_global}"
This prints:
in a class: 0
in a method: 0
at top-level, $global: 1, $other_global: 3
An uninitialized global variable has a value of nil+.
Ruby has some special globals that behave differently depending on context
such as the regular expression match variables or that have a side-effect when
assigned to. See the {global variables documentation}[rdoc-ref:language/globals.md]
for details.

Contoh Kode / Perintah CLI:

```
def my_method
puts "in a method: #{$global}"
$global = $global + 1
$other_global = 3
end
```

[ruby_035] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Assignment Methods

Sumber: ht

== Assignment Methods
You can define methods that will behave like assignment, for example:
class C
def value=(value)
@value = value
end
end
c = C.new
c.value = 42
Using assignment methods allows your programs to look nicer. When assigning
to an instance variable most people use Module#attr_accessor:

class C
attr_accessor :value
end
When using method assignment you must always have a receiver. If you do not have a receiver, Ruby assumes you are assigning to a local variable:
class C
attr_accessor :value
def my_method
value = 42
puts "local_variables: #{local_variables.join ", "}"
puts "@value: #{@value.inspect}"
end
end
C.new.my_method
This prints:
local_variables: value
@value: nil
To use the assignment method you must set the receiver:
class C
attr_accessor :value
def my_method
self.value = 42
puts "local_variables: #{local_variables.join ", "}"
puts "@value: #{@value.inspect}"
end
end
C.new.my_method
This prints:
local_variables:
@value: 42
Note that the value returned by an assignment method is ignored whatever, since an assignment expression result is always the assignment value.

Contoh Kode / Perintah CLI:

```
def my_method
value = 42
puts "local_variables: #{local_variables.join ", "}"
puts "@value: #{@value.inspect}"
end

# ----------------------------------------

def my_method
self.value = 42
puts "local_variables: #{local_variables.join ", "}"
puts "@value: #{@value.inspect}"
end
```

[ruby_036] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Abbreviated Assignment

Sumber: ht

== Abbreviated Assignment
You can mix several of the operators and assignment. To add 1 to an object
you can write:
a = 1
a = 2
p a # prints 3
This is equivalent to:
a = 1
a = a 2
p a # prints 3
You can use the following operators this way: , -,
*, /, %, **,
&, |, ^, ,
>>
There are also ||= and &&=. The former makes an
assignment if the value was nil or false while the latter makes an
assignment if the value was not nil or false+.
Here is an example:
a ||= 0
a &&= 1
p a # prints 1
Note that these two operators behave more like a || a = 0 than
a = a || 0.

[ruby_037] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Implicit Array Assignment

Sumber: ht

== Implicit Array Assignment
You can implicitly create an array by listing multiple values when assigning:
a = 1, 2, 3
p a # prints [1, 2, 3]
This implicitly creates an Array.
You can use * or the "splat" operator or unpack an Array when
assigning. This is similar to multiple assignment:
a = *[1, 2, 3]
p a # prints [1, 2, 3]
b = *1
p b # prints [1]
You can splat anywhere in the right-hand side of the assignment:
a = 1, *[2, 3]
p a # prints [1, 2, 3]

[ruby_038] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Multiple Assignment

Sumber: ht

```
== Multiple Assignment
You can assign multiple values on the right-hand side to multiple variables:
a, b = 1, 2
p a: a, b: b # prints {:a=>1, :b=>2}
In the following sections any place "variable" is used an assignment method,
instance, class or global will also work:
def value=(value)
p assigned: value
end
self.value, $global = 1, 2 # prints {:assigned=>1}
p $global # prints 2
You can use multiple assignment to swap two values in-place:
old_value = 1
new_value, old_value = old_value, 2
p new_value: new_value, old_value: old_value
# prints {:new_value=>1, :old_value=>2}
If you have more values on the right hand side of the assignment than variables
on the left hand side, the extra values are ignored:
a, b = 1, 2, 3
p a: a, b: b # prints {:a=>1, :b=>2}
You can use * to gather extra values on the right-hand side of
the assignment.
a, *b = 1, 2, 3
p a: a, b: b # prints {:a=>1, :b=>[2, 3]}
The * can appear anywhere on the left-hand side:
*a, b = 1, 2, 3
p a: a, b: b # prints {:a=>[1, 2], :b=>3}
But you may only use one * in an assignment.
```

[ruby_039] Ruby - Assignment & Variables

Assignment (Local, Instance @, Class @@, Global $, Multiple assignment): Array Decomposition

Sumber: ht

```
== Array Decomposition
Like Array decomposition in {method arguments}[rdoc-ref:syntax/methods.rdoc]
you can decompose an Array during assignment using parenthesis:
(a, b) = [1, 2]
p a: a, b: b # prints {:a=>1, :b=>2}
You can decompose an Array as part of a larger multiple assignment:
a, (b, c) = 1, [2, 3]
p a: a, b: b, c: c # prints {:a=>1, :b=>2, :c=>3}
Since each decomposition is considered its own multiple assignment you can use
* to gather arguments in the decomposition:
a, (b, *c), *d = 1, [2, 3, 4], 5, 6
p a: a, b: b, c: c, d: d
# prints {:a=>1, :b=>2, :c=>[3, 4], :d=>[5, 6]}
```

[ruby_040] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): Overview

Sumber: ht

= Control Expressions
Ruby has a variety of ways to control execution. All the expressions described
here return a value.
For the tests in these control expressions, nil and false are false-values
and true and any other object are true-values. In this document "true" will
mean "true-value" and "false" will mean "false-value".

[ruby_041] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +if+ Expression

Sumber: ht

== if Expression
The simplest if expression has two parts, a "test" expression and a "then"
expression. If the "test" expression evaluates to a true then the "then"
expression is evaluated.
Here is a simple if statement:
if true then
puts "the test resulted in a true-value"
end
This will print "the test resulted in a true-value".
The then is optional:
if true
puts "the test resulted in a true-value"
end
This document will omit the optional then for all expressions as that is the
most common usage of if.
You may also add an else expression. If the test does not evaluate to true
the else expression will be executed:
if false
puts "the test resulted in a true-value"
else
puts "the test resulted in a false-value"
end
This will print "the test resulted in a false-value".
You may add an arbitrary number of extra tests to an if expression using
elsif. An elsif executes when all tests above the elsif are false.
a = 1
if a == 0
puts "a is zero"
elsif a == 1
puts "a is one"
else
puts "a is some other value"
end
This will print "a is one" as 1 is not equal to 0.
Since else is only executed when there are no matching conditions.
Once a condition matches, either the if condition or any elsif condition,
the if expression is complete and no further tests will be performed.
Like an if, an elsif condition may be followed by a then.

In this example only "a is one" is printed:
a = 1
if a == 0
puts "a is zero"
elsif a == 1
puts "a is one"
elsif a >= 1
puts "a is greater than or equal to one"
else
puts "a is some other value"
end
The tests for if and elsif may have side-effects. The most common use of side-effect is to cache a value into a local variable:
if a = object.some_value
do something to a
end
The result value of an if expression is the last value executed in the expression.

[ruby_042] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): Ternary if

Sumber: ht

== Ternary if
You may also write a if-then-else expression using ? and :. This ternary if:
input_type = gets =~ /hello/i ? "greeting" : "other"
Is the same as this if expression:
input_type =
if gets =~ /hello/i
"greeting"
else
"other"
end
While the ternary if is much shorter to write than the more verbose form, for readability it is recommended that the ternary if is only used for simple conditionals. Also, avoid using multiple ternary conditions in the same expression as this can be confusing.

[ruby_043] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +unless+ Expression

Sumber: ht

== unless Expression
The unless expression is the opposite of the if expression. If the value is false, the "then" expression is executed:
unless true
puts "the value is a false-value"
end
This prints nothing as true is not a false-value.
You may use an optional then with unless just like if.

Note that the above unless expression is the same as:
if not true
puts "the value is a false-value"
end
Like an if expression you may use an else condition with unless:
unless true
puts "the value is false"
else
puts "the value is true"
end
This prints "the value is true" from the else condition.
You may not use elsif with an unless expression.
The result value of an unless expression is the last value executed in the expression.

[ruby_044] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): Modifier +if+ and +unless+

Sumber: ht

== Modifier if and unless
if and unless can also be used to modify an expression. When used as a modifier the left-hand side is the "then" statement and the right-hand side is the "test" expression:
a = 0
a = 1 if a.zero?
p a
This will print 1.
a = 0
a = 1 unless a.zero?
p a
This will print 0.
While the modifier and standard versions have both a "test" expression and a "then" statement, they are not exact transformations of each other due to parse order. Here is an example that shows the difference:
p a if a = 0.zero?
This raises the NameError "undefined local variable or method `a'".
When ruby parses this expression it first encounters a as a method call in the "then" expression, then later it sees the assignment to a in the "test" expression and marks a as a local variable.
When running this line it first executes the "test" expression, a = 0.zero?.
Since the test is true it executes the "then" expression, p a.
Since the a in the body was recorded as a method which does not exist the NameError is raised.
The same is true for unless.

[ruby_045] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +case+ Expression

Sumber: ht

== case Expression
The case expression can be used in two ways.
The most common way is to compare an object against multiple patterns. The
patterns are matched using the === method which is aliased to == on
Object. Other classes must override it to give meaningful behavior. See
Module#=== and Regexp#=== for examples.
Here is an example of using case to compare a String against a pattern:
case "12345"
when /^1/
puts "the string starts with one"
else
puts "I don't know what the string starts with"
end
Here the string "12345" is compared with /^1/ by
calling /^1/ === "12345" which returns true. Like the if
expression, the first when that matches is executed and all other matches are
ignored.
If no matches are found, the else is executed.
The else and then are optional, this case expression gives the same
result as the one above:
case "12345"
when /^1/
puts "the string starts with one"
end
You may place multiple conditions on the same when:
case "2"
when /^1/, "2"
puts "the string starts with one or is '2'"
end
Ruby will try each condition in turn, so first /^1/ === "2"
returns false, then "2" === "2" returns true, so "the string
starts with one or is '2'" is printed.
You may use then after the when condition. This is most frequently used
to place the body of the when on a single line.
case a
when 1, 2 then puts "a is one or two"
when 3 then puts "a is three"
else puts "I don't know what a is"
end
The other way to use a case expression is like an if-elsif expression:
a = 2
case
when a == 1, a == 2
puts "a is one or two"
when a == 3
puts "a is three"
else

puts "I don't know what a is"
end
Again, the then and else are optional.
The result value of a case expression is the last value executed in the
expression.
Since Ruby 2.7, case expressions also provide a

[ruby_046] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +while+ Loop

Sumber: ht

== while Loop
The while loop executes while a condition is true:
a = 0
while a a is checked
before the loop is entered, then the body executes, then the condition is
checked again. When the condition results in false the loop is terminated.
The do keyword is optional. The following loop is equivalent to the loop
above:
while a < 10
p a
a = 1
end
The result of a while loop is nil unless break+ is used to supply a
value.

[ruby_047] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +until+ Loop

Sumber: ht

== until Loop
The until loop executes while a condition is false:
a = 0
until a > 10 do
p a
a = 1
end
p a
This prints the numbers 0 through 11. Like a while loop the condition a
> 10 is checked when entering the loop and each time the loop body
executes. If the condition is false the loop will continue to execute.
Like a while loop, the do is optional.
Like a while loop, the result of an until loop is nil unless break+ is
used.

[ruby_048] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +for+ Loop

Sumber: ht

== for Loop
The for loop consists of for followed by a variable to contain the
iteration argument followed by in and the value to iterate over using #each.
The do is optional:
for value in [1, 2, 3] do
puts value
end
Prints 1, 2 and 3.
Like while and until, the do is optional.
The for loop is similar to using #each, but does not create a new variable
scope.
The result value of a for loop is the value iterated over unless break is
used.
The for loop is rarely used in modern ruby programs.

[ruby_049] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): Modifier +while+ and +until+

Sumber: ht

== Modifier while and until
Like if and unless, while and until can be used as modifiers:
a = 0
a = 1 while a 10
p a # prints 11
You can use begin and end to create a while loop that runs the body once
before the condition:
a = 0
begin
a = 1
end while a < 10
p a # prints 10
If you don't use rescue or ensure, Ruby optimizes away any exception
handling overhead.

[ruby_050] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +break+ Statement

Sumber: ht

== break Statement
Use break to leave a block early. This will stop iterating over the items in values if one of them is even:
values.each do |value|
break if value.even?
...
end
You can also terminate from a while loop using break:
a = 0
while true do

p a
a = 1
break if a < 10
end
p a
This prints the numbers 0 and 1.
break+ accepts a value that supplies the result of the expression it is
"breaking" out of:
result = [1, 2, 3].each do |value|
break value * 2 if value.even?
end
p result # prints 4

[ruby_051] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +next+ Statement

Sumber: ht

== next Statement
Use next to skip the rest of the current iteration:
result = [1, 2, 3].map do |value|
next if value.even?
value * 2
end
p result # prints [2, nil, 6]
next accepts an argument that can be used as the result of the current
block iteration:
result = [1, 2, 3].map do |value|
next value if value.even?
value * 2
end
p result # prints [2, 2, 6]

[ruby_052] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): +redo+ Statement

Sumber: ht

== redo Statement
Use redo to redo the current iteration:
result = []
while result.length < 10 do
result << result.length
redo if result.last.even?
result << result.length 1
end
p result
This prints [0, 1, 3, 3, 5, 5, 7, 7, 9, 9, 11]
In Ruby 1.8, you could also use retry where you used redo. This is no
longer true, now you will receive a SyntaxError when you use retry outside
of a rescue block. See {Exceptions}[rdoc-ref:syntax/exceptions.rdoc]
for proper usage of retry+.

[ruby_053] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): Modifier Statements

Sumber: ht

== Modifier Statements
Ruby's grammar differentiates between statements and expressions. All
expressions are statements (an expression is a type of statement), but
not all statements are expressions. Some parts of the grammar accept
expressions and not other types of statements, which causes code that
looks similar to be parsed differently.
For example, when not used as a modifier, if, else, while, until,
and begin are expressions (and also statements). However, when
used as a modifier, if, else, while, until and rescue
are statements but not expressions.
if true; 1 end # expression (and therefore statement)
1 if true # statement (not expression)
Statements that are not expressions cannot be used in contexts where an
expression is expected, such as method arguments.
puts(1 if true) #=> SyntaxError
You can wrap a statement in parentheses to create an expression.
puts((1 if true)) #=> 1
If you put a space between the method name and opening parenthesis, you
do not need two sets of parentheses.
puts (1 if true) #=> 1, because of optional parentheses for method
This is because this is parsed similar to a method call without
parentheses. It is equivalent to the following code, without the creation
of a local variable:
x = (1 if true)
p x
In a modifier statement, the left-hand side must be a statement and the
right-hand side must be an expression.
So in a if b rescue c, because b rescue c is a
statement that is not an expression, and therefore is not allowed as the
right-hand side of the if modifier statement, the code is necessarily
parsed as (a if b) rescue c.
This interacts with operator precedence in such a way that:
stmt if v = expr rescue x
stmt if v = expr unless x
are parsed as:
stmt if v = (expr rescue x)
(stmt if v = expr) unless x
This is

[ruby_054] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): Flip-Flop

Sumber: ht

== Flip-Flop
The flip-flop is a slightly special conditional expression. One of its
typical uses is processing text from ruby one-line programs used with
ruby -n or ruby -p.
The form of the flip-flop is an expression that indicates when the

flip-flop turns on, .. (or ...), then an expression
that indicates when the flip-flop will turn off. While the flip-flop is on it
will continue to evaluate to true, and false when off.
Here is an example:
selected = []
0.upto 10 do |value|
selected n==2. The flip-flop
is initially `off' (false) for 0 and 1, but becomes `on' (true) for 2 and
remains `on' through 8.  After 8 it turns off and remains `off' for 9 and 10.
The flip-flop must be used inside a conditional such as !,
? :, not, if, while, unless, until etc. including the
modifier forms.
When you use an inclusive range (..), the `off' condition is
evaluated when the `on' condition changes:
selected = []
0.upto 5 do |value|
selected ...), the `off' condition is
evaluated on the following iteration:
selected = []
0.upto 5 do |value|
selected << value if value==2...value==2
end
p selected # prints [2, 3, 4, 5]
Here, the flip-flop turns on when value equals 2, but doesn't turn off on the
same iteration. The `off' condition isn't evaluated until the following
iteration and value will never be two again.

[ruby_055] Ruby - Control Expressions

Control Expressions (if/unless, case/when, while/until, break/next/redo): throw/catch

Sumber: ht

== throw/catch
throw and catch are used to implement non-local control flow in Ruby. They
operate similarly to exceptions, allowing control to pass directly from the
place where throw is called to the place where the matching catch is
called. The main difference between throw/catch and the use of exceptions
is that throw/catch are designed for expected non-local control flow,
while exceptions are designed for exceptional control flow situations, such
as handling unexpected errors.
When using throw, you provide 1-2 arguments. The first argument is the
value for the matching catch. The second argument is optional (defaults to
nil), and will be the value that catch returns if there is a matching
throw inside the catch block. If no matching throw method is called
inside a catch block, the catch method returns the return value of the
block passed to it.
def a(n)
throw :d, :a if n == 0
b(n)
end
def b(n)
throw :d, :b if n == 1
c(n)

end
def c(n)
throw :d if n == 2
end
4.times.map do |i|
catch(:d) do
a(i)
:default
end
end
=> [:a, :b, nil, :default]
If the first argument you pass to throw is not handled by a matching catch, an UncaughtThrowError exception will be raised. This is because throw/catch should only be used for expected control flow changes, so using a value that is not already expected is an error.
throw/catch are implemented as Kernel methods (Kernel#throw and Kernel#catch), not as keywords. So they are not usable directly if you are in a BasicObject context. You can use Kernel.throw and Kernel.catch in this case:
BasicObject.new.instance_exec do
def a
b
end
def b
c
end
def c
::Kernel.throw :d, :e
end
result = ::Kernel.catch(:d) do
a
end
result # => :e
end

Contoh Kode / Perintah CLI:

```
def b
c
end
def c
::Kernel.throw :d, :e
end
result = ::Kernel.catch(:d) do
a
end
result # => :e
```

[ruby_056] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Overview

Sumber: ht

= Methods
Methods implement the functionality of your program. Here is a simple method
definition:
def one_plus_one
1 1
end
A method definition consists of the def keyword, a method name, the body of
the method, return value and the end keyword. When called the method will
execute the body of the method. This method returns 2.
Since Ruby 3.0, there is also a shorthand syntax for methods consisting
of exactly one expression:
def one_plus_one = 1 1
This section only covers defining methods. See also the {syntax documentation
on calling methods}[rdoc-ref:syntax/calling_methods.rdoc].

[ruby_057] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Method Names

Sumber: ht

== Method Names
Method names may be one of the operators or must start a letter or a character
with the eighth bit set. It may contain letters, numbers, an _
(underscore or low line) or a character with the eighth bit set. The convention
is to use underscores to separate words in a multiword method name:
def method_name
puts "use underscores to separate words"
end
Ruby programs must be written in a US-ASCII-compatible character set such as
UTF-8, ISO-8859-1 etc. In such character sets if the eighth bit is set it
indicates an extended character. Ruby allows method names and other identifiers
to contain such characters. Ruby programs cannot contain some characters like
ASCII NUL (\x00).
The following are examples of valid Ruby methods:
def hello
"hello"
end
def ?????
puts "means hello in Japanese"
end
Typically method names are US-ASCII compatible since the keys to type them
exist on all keyboards.
Method names may end with a ! (bang or exclamation mark), a
? (question mark), or = (equals sign).
The bang methods (! at the end of the method name) are called and
executed just like any other method. However, by convention, a method with an
exclamation point or bang is considered dangerous. In Ruby's core library the

dangerous method implies that when a method ends with a bang (!),
it indicates that unlike its non-bang equivalent, permanently modifies its
receiver. Almost always, the Ruby core library will have a non-bang
counterpart (method name which does NOT end with !) of every bang
method (method name which does end with !) that does not modify
the receiver. This convention is typically true for the Ruby core library but
may or may not hold true for other Ruby libraries.
Methods that end with a question mark by convention return boolean, but they
may not always return ju

Contoh Kode / Perintah CLI:

```
def attr=(val)
@attr = val
end

# ----------------------------------------

def []=(a, b, c)
puts a * b + c
end
```

[ruby_058] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Return Values

Sumber: ht

== Return Values
By default, a method returns the last expression that was evaluated in the body
of the method. In the example above, the last (and only) expression evaluated
was the simple sum 1 1. The return keyword can be used to
make it explicit that a method returns a value.
def one_plus_one
return 1 1
end
It can also be used to make a method return before the last expression is
evaluated.
def two_plus_two
return 2 2
1 1 # this expression is never evaluated
end
Note that for assignment methods the return value will be ignored when using
the assignment syntax. Instead, the argument will be returned:
def a=(value)
return 1 + value
end
p(self.a = 5) # prints 5
The actual return value will be returned when invoking the method directly:
p send(:a=, 5) # prints 6

[ruby_059] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Scope

Sumber: ht

== Scope
The standard syntax to define a method:
def my_method
...
end
adds the method to a class. You can define an instance method on a specific
class with the class keyword:
class C
def my_method
...
end
end
A method may be defined on another object. You may define a "class method" (a
method that is defined on the class, not an instance of the class) like this:
class C
def self.my_method
...
end
end
However, this is simply a special case of a greater syntactical power in Ruby,
the ability to add methods to any object. Classes are objects, so adding
class methods is simply adding methods to the Class object.
The syntax for adding a method to an object is as follows:
greeting = "Hello"
def greeting.broaden
self ", world!"
end
greeting.broaden # returns "Hello, world!"
self is a keyword referring to the current object under consideration
by the compiler, which might make the use of self in defining a class
method above a little clearer. Indeed, the example of adding a hello
method to the class String can be rewritten thus:
def String.hello
"Hello, world!"
end
A method defined like this is called a "singleton method". broaden will only
exist on the string instance greeting. Other strings will not have broaden+.

[ruby_060] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Overriding

Sumber: ht

== Overriding
When Ruby encounters the def keyword, it doesn't consider it an error if the
method already exists: it simply redefines it. This is called
overriding. Rather like extending core classes, this is a potentially

dangerous ability, and should be used sparingly because it can cause unexpected
results. For example, consider this irb session:
>> "43".to_i
=> 43
>> class String
>>   def to_i
>>   42
>>   end
>> end
=> nil
>> "43".to_i
=> 42
This will effectively sabotage any code which makes use of the method
String#to_i to parse numbers from strings.

[ruby_061] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Arguments

Sumber: ht

== Arguments
A method may accept arguments. The argument list follows the method name:
def add_one(value)
value 1
end
When called, the user of the add_one method must provide an argument. The
argument is a local variable in the method body. The method will then add one
to this argument and return the value. If given 1 this method will
return 2.
The parentheses around the arguments are optional:
def add_one value
value 1
end
The parentheses are mandatory in shorthand method definitions:
OK
def add_one(value) = value 1
SyntaxError
def add_one value = value 1
Multiple arguments are separated by a comma:
def add_values(a, b)
a + b
end
When called, the arguments must be provided in the exact order. In other
words, the arguments are positional.

[ruby_062] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Default Values

Sumber: ht

=== Default Values
Arguments may have default values:
def add_values(a, b = 1)
a b
end
The default value does not need to appear first, but arguments with defaults
must be grouped together. This is ok:
def add_values(a = 1, b = 2, c)
a b c
end
This will raise a SyntaxError:
def add_values(a = 1, b, c = 1)
a b c
end
Default argument values can refer to arguments that have already been
evaluated as local variables, and argument values are always evaluated
left to right. So this is allowed:
def add_values(a = 1, b = a)
a b
end
add_values
=> 2
But this will raise a NameError (unless there is a method named
b defined):
def add_values(a = b, b = 1)
a + b
end
add_values
NameError (undefined local variable or method `b' for main:Object)

[ruby_063] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Array Decomposition

Sumber: ht

=== Array Decomposition
You can decompose (unpack or extract values from) an Array using extra
parentheses in the arguments:
def my_method((a, b))
p a: a, b: b
end
my_method([1, 2])
This prints:
{:a=>1, :b=>2}
If the argument has extra elements in the Array they will be ignored:
def my_method((a, b))
p a: a, b: b

```
end
my_method([1, 2, 3])
```

This has the same output as above.

You can use a * to collect the remaining arguments. This splits an Array into a first element and the rest:

```
def my_method((a, *b))
p a: a, b: b
end
my_method([1, 2, 3])
```

This prints:

```
{:a=>1, :b=>[2, 3]}
```

The argument will be decomposed if it responds to #to_ary. You should only define #to_ary if you can use your object in place of an Array.

Use of the inner parentheses only uses one of the sent arguments. If the argument is not an Array it will be assigned to the first argument in the decomposition and the remaining arguments in the decomposition will be nil:

```
def my_method(a, (b, c), d)
p a: a, b: b, c: c, d: d
end
my_method(1, 2, 3)
```

This prints:

```
{:a=>1, :b=>2, :c=>nil, :d=>3}
```

You can nest decomposition arbitrarily:

```
def my_method(((a, b), c))
# ...
end
```

[ruby_064] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Array/Hash Argument

Sumber: ht

=== Array/Hash Argument

Prefixing an argument with * causes any remaining arguments to be converted to an Array:

```
def gather_arguments(*arguments)
p arguments
end
gather_arguments 1, 2, 3 # prints [1, 2, 3]
```

The array argument must appear before any keyword arguments.

It is possible to gather arguments at the beginning or in the middle:

```
def gather_arguments(first_arg, *middle_arguments, last_arg)
p middle_arguments
end
gather_arguments 1, 2, 3, 4 # prints [2, 3]
```

The array argument will capture a Hash as the last entry if keywords were provided by the caller after all positional arguments.

```
def gather_arguments(*arguments)
p arguments
end
gather_arguments 1, a: 2 # prints [1, {:a=>2}]
```

However, this only occurs if the method does not declare any keyword arguments.

```
def gather_arguments_keyword(*positional, keyword: nil)
p positional: positional, keyword: keyword
end
gather_arguments_keyword 1, 2, three: 3
#=> raises: unknown keyword: three (ArgumentError)
```

Also, note that a bare * can be used to ignore arguments:

```
def ignore_arguments(*)
end
```

You can also use a bare * when calling a method to pass the arguments directly to another method:

```
def delegate_arguments(*)
other_method(*)
end
```

[ruby_065] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Keyword Arguments

Sumber: ht

=== Keyword Arguments

Keyword arguments are similar to positional arguments with default values:

```
def add_values(first: 1, second: 2)
first  second
end
```

Arbitrary keyword arguments will be accepted with **:

```
def gather_arguments(first: nil, **rest)
p first, rest
end
gather_arguments first: 1, second: 2, third: 3
# prints 1 then {:second=>2, :third=>3}
```

When calling a method with keyword arguments the arguments may appear in any order. If an unknown keyword argument is sent by the caller, and the method does not accept arbitrary keyword arguments, an ArgumentError is raised.

To require a specific keyword argument, do not include a default value for the keyword argument:

```
def add_values(first:, second:)
first  second
end
add_values
# ArgumentError (missing keywords: first, second)
add_values(first: 1, second: 2)
# => 3
```

When mixing keyword arguments and positional arguments, all positional arguments must appear before any keyword arguments.

Also, note that ** can be used to ignore keyword arguments:

```
def ignore_keywords(**)
end
```

You can also use ** when calling a method to delegate keyword arguments to another method:

```
def delegate_keywords(**)
other_method(**)
end
```

To mark a method as accepting keywords, but not actually accepting
keywords, you can use the **nil:
def no_keywords(**nil)
end
Calling such a method with keywords or a non-empty keyword splat will
result in an ArgumentError. This syntax is supported so that keywords
can be added to the method later without affected backwards compatibility.
If a method definition does not accept any keywords, and the
**nil syntax is not used, any keywords provided when calling
the method will be converted to a Hash positional argument:
def meth(arg)
arg
end
meth(a: 1)
=> {:a=>1}

[ruby_066] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Block Argument

Sumber: ht

=== Block Argument
The block argument is indicated by & and must come last:
def my_method(&my_block)
my_block.call(self)
end
Most frequently the block argument is used to pass a block to another method:
def each_item(&block)
@items.each(&block)
end
You are not required to give a name to the block if you will just be passing
it to another method:
def each_item(&)
@items.each(&)
end
If you are only going to call the block and will not otherwise manipulate it
or send it to another method, using yield without an explicit
block parameter is preferred. This method is equivalent to the first method
in this section:
def my_method
yield self
end

[ruby_067] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Argument Forwarding

Sumber: ht

=== Argument Forwarding
Since Ruby 2.7, an all-arguments forwarding syntax is available:
def concrete_method(*positional_args, **keyword_args, &block)
[positional_args, keyword_args, block]

```
end
def forwarding_method(...)
concrete_method(...)
end
forwarding_method(1, b: 2) { puts 3 }
#=>  [[1], {:b=>2}, #]
Calling with forwarding ... is available only in methods
defined with ....
def regular_method(arg, **kwarg)
concrete_method(...) # Syntax error
end
Since Ruby 3.0, there can be leading arguments before ...
both in definitions and in invocations (but in definitions they can be
only positional arguments without default values).
def request(method, path, **headers)
puts "#{method.upcase} #{path} #{headers}"
end
def get(...)
request(:GET, ...) # leading argument in invoking
end
get('http://ruby-lang.org', 'Accept' => 'text/html')
# Prints: GET http://ruby-lang.org {"Accept"=>"text/html"}
def logged_get(msg, ...) # leading argument in definition
puts "Invoking #get: #{msg}"
get(...)
end
logged_get('Ruby site', 'http://ruby-lang.org')
# Prints:
#   Invoking #get: Ruby site
#   GET http://ruby-lang.org {}
Note that omitting parentheses in forwarding calls may lead to
unexpected results:
def log(...)
puts ...  # This would be treated as `puts()...',
# i.e. endless range from puts result
end
log("test")
# Prints: warning: ... at EOL, should be parenthesized?
# ...and then empty line
```

[ruby_068] Ruby - Methods & Modern Syntax

Methods (Default args, Keyword args **, Endless methods def foo() = expr, Blocks): Exception Handling

Sumber: ht

```
== Exception Handling
Methods have an implied exception handling block so you do not need to use
begin or end to handle exceptions.  This:
def my_method
begin
# code that may raise an exception
rescue
```

handle exception
end
end
May be written as:
def my_method
code that may raise an exception
rescue
handle exception
end
Similarly, if you wish to always run code even if an exception is raised,
you can use ensure without begin and end:
def my_method
code that may raise an exception
ensure
code that runs even if previous code raised an exception
end
You can also combine rescue with ensure and/or else, without
begin and end:
def my_method
code that may raise an exception
rescue
handle exception
else
only run if no exception raised above
ensure
code that runs even if previous code raised an exception
end
If you wish to rescue an exception for only part of your method, use begin and
end. For more details see the page on {exception
handling}[rdoc-ref:syntax/exceptions.rdoc].

[ruby_069] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Overview

Sumber: ht

= Pattern matching
Pattern matching is a feature allowing deep matching of structured values: checking the structure and binding the matched parts to local variables.
Pattern matching in Ruby is implemented with the case/in expression:
case
in
...
in
...
in
...
else
...
end
(Note that in and when branches can NOT be mixed in one case expression.)
Or with the => operator and the in operator, which can be used in a standalone expression:

=>
in
The case/in expression is _exhaustive_: if the value of the expression does not match any branch of the case expression (and the else branch is absent), NoMatchingPatternError is raised.
Therefore, the case expression might be used for conditional matching and unpacking:
config = {db: {user: 'admin', password: 'abc123'}}
case config
in db: {user:} # matches subhash and puts matched value in variable user
puts "Connect with user '#{user}'"
in connection: {username: }
puts "Connect with user '#{username}'"
else
puts "Unrecognized structure of config"
end
Prints: "Connect with user 'admin'"
whilst the => operator is most useful when the expected data structure is known beforehand, to just unpack parts of it:
config = {db: {user: 'admin', password: 'abc123'}}
config => {db: {user:}} # will raise if the config's structure is unexpected
puts "Connect with user '#{user}'"
Prints: "Connect with user 'admin'"
in is the same as case ; in ; true; else false; end.
You can use it when you only want to know if a pattern has been matched or not:
users = [{name: "Alice", age: 12}, {name: "Bob", age: 23}]
users.any? {|user| user in {name: /B/, age: 20..} } #=> true
See below for more examples and explanations of the syntax.

Contoh Kode / Perintah CLI:

```
case <expression>
in <pattern1>
...
in <pattern2>
...
in <pattern3>
...
else
...
end

# ----------------------------------------

<expression> => <pattern>
<expression> in <pattern>
```

[ruby_070] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Patterns

Sumber: ht

== Patterns
Patterns can be:
* any Ruby object (matched by the === operator, like in when); (Value pattern)
* array pattern: [, , , ...]; (Array pattern)
* find pattern: [*variable, , , , ..., *variable]; (Find pattern)
* hash pattern: {key: , key: , ...}; (Hash pattern)
* combination of patterns with |; (Alternative pattern)

* variable capture: => variable or variable; (As pattern, Variable pattern)
Any pattern can be nested inside array/find/hash patterns where is specified.
Array patterns and find patterns match arrays, or objects that respond to deconstruct (see below about the latter).
Hash patterns match hashes, or objects that respond to deconstruct_keys (see below about the latter). Note that only symbol keys are supported for hash patterns.
An important difference between array and hash pattern behavior is that arrays match only a _whole_ array:
case [1, 2, 3]
in [Integer, Integer]
"matched"
else
"not matched"
end
#=> "not matched"
while the hash matches even if there are other keys besides the specified part:
case {a: 1, b: 2, c: 3}
in {a: Integer}
"matched"
else
"not matched"
end
#=> "matched"
{} is the only exclusion from this rule. It matches only if an empty hash is given:
case {a: 1, b: 2, c: 3}
in {}
"matched"
else
"not matched"
end
#=> "not matched"
case {}
in {}
"matched"
else
"not matched"
end
#=> "matched"
There is also a way to specify there should be no other keys in the matched hash except those explicitly specified by the pattern, with **nil:
case {a: 1, b: 2}
in {a: Integer, **nil} # this will not match the pattern having keys other than a:
"matched a part"
in {a: Integer, b: Integer, **nil}
"matched a whole"
else
"not matched"
end
#=> "matched a whole"
Both array and hash patterns support "rest" specification:
case [1, 2, 3]
in [Integer, *]
"

[ruby_071] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Variable binding

Sumber: ht

== Variable binding
Besides deep structural checks, one of the very important features of the pattern matching is the binding of the matched parts to local variables. The basic form of binding is just specifying => variable_name after the matched (sub)pattern (one might find this similar to storing exceptions in local variables in a rescue ExceptionClass => var clause):

```
case [1, 2]
in Integer => a, Integer
"matched: #{a}"
else
"not matched"
end
#=> "matched: 1"
case {a: 1, b: 2, c: 3}
in a: Integer => m
"matched: #{m}"
else
"not matched"
end
#=> "matched: 1"
```

If no additional check is required, for only binding some part of the data to a variable, a simpler form could be used:

```
case [1, 2]
in a, Integer
"matched: #{a}"
else
"not matched"
end
#=> "matched: 1"
case {a: 1, b: 2, c: 3}
in a: m
"matched: #{m}"
else
"not matched"
end
#=> "matched: 1"
```

For hash patterns, even a simpler form exists: key-only specification (without any sub-pattern) binds the local variable with the key's name, too:

```
case {a: 1, b: 2, c: 3}
in a:
"matched: #{a}"
else
"not matched"
end
#=> "matched: 1"
```

\Binding works for nested patterns as well:

```
case {name: 'John', friends: [{name: 'Jane'}, {name: 'Rajesh'}]}
in name:, friends: [{name: first_friend}, *]
"matched: #{first_friend}"
else
```

```
"not matched"
end
#=> "matched: Jane"
```

The "rest" part of a pattern also can be bound to a variable:

```
case [1, 2, 3]
in a, *rest
"matched: #{a}, #{rest}"
else
"not matched"
end
#=> "matched: 1, [2, 3]"
case {a: 1, b: 2, c: 3}
in a:, **rest
"matched: #{a}, #{rest}"
else
"not matched"
end
#=> "matched: 1, {b: 2, c: 3}"
```

\Binding to variables currently does NOT work for alternative patterns joined with |:

```
case {a: 1, b: 2}
in {a: } | Array
# ^ SyntaxError (variable capture in alternative pattern)
"matched: #{a}"
else
"not matc
```

[ruby_072] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Variable pinning

Sumber: ht

== Variable pinning

Due to the variable binding feature, existing local variable can not be straightforwardly used as a sub-pattern:

```
expectation = 18
case [1, 2]
in expectation, *rest
"matched. expectation was: #{expectation}"
else
"not matched. expectation was: #{expectation}"
end
# expected: "not matched. expectation was: 18"
# real: "matched. expectation was: 1" -- local variable just rewritten
```

For this case, the pin operator ^ can be used, to tell Ruby "just use this value as part of the pattern":

```
expectation = 18
case [1, 2]
in ^expectation, *rest
"matched. expectation was: #{expectation}"
else
"not matched. expectation was: #{expectation}"
end
#=> "not matched. expectation was: 18"
```

One important usage of variable pinning is specifying that the same value should occur in the pattern several times:

```
jane = {school: 'high', schools: [{id: 1, level: 'middle'}, {id: 2, level: 'high'}]}
john = {school: 'high', schools: [{id: 1, level: 'middle'}]}
case jane
in school:, schools: [*, {id:, level: ^school}] # select the last school, level should match
"matched. school: #{id}"
else
"not matched"
end
#=> "matched. school: 2"
case john # the specified school level is "high", but last school does not match
in school:, schools: [*, {id:, level: ^school}]
"matched. school: #{id}"
else
"not matched"
end
#=> "not matched"
```

In addition to pinning local variables, you can also pin instance, global, and class variables:

```
$gvar = 1
class A
@ivar = 2
@@cvar = 3
case [1, 2, 3]
in ^$gvar, ^@ivar, ^@@cvar
"matched"
else
"not matched"
end
#=> "matched"
end
```

You can also pin the result of arbitrary expressions using parentheses:

```
a = 1
b = 2
case 3
in ^(a + b)
"matched"
else
"not matched"
end
#=> "matched"
```

[ruby_073] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Matching non-primitive objects: +deconstruct+ and +deconstruct_keys+

Sumber: ht

== Matching non-primitive objects: deconstruct and deconstruct_keys

As already mentioned above, array, find, and hash patterns besides literal arrays and hashes will try to match any object implementing deconstruct (for array/find patterns) or deconstruct_keys (for hash patterns).

```
class Point
def initialize(x, y)
@x, @y = x, y
end
```

```
def deconstruct
puts "deconstruct called"
[@x, @y]
end
def deconstruct_keys(keys)
puts "deconstruct_keys called with #{keys.inspect}"
{x: @x, y: @y}
end
end
case Point.new(1, -2)
in px, Integer  # sub-patterns and variable binding works
"matched: #{px}"
else
"not matched"
end
# prints "deconstruct called"
"matched: 1"
case Point.new(1, -2)
in x: 0.. => px
"matched: #{px}"
else
"not matched"
end
# prints: deconstruct_keys called with [:x]
#=> "matched: 1"
```

keys are passed to deconstruct_keys to provide a room for optimization in the matched class: if calculating a full hash representation is expensive, one may calculate only the necessary subhash. When the **rest pattern is used, nil is passed as a keys value:

```
case Point.new(1, -2)
in x: 0.. => px, **rest
"matched: #{px}"
else
"not matched"
end
# prints: deconstruct_keys called with nil
#=> "matched: 1"
```

Additionally, when matching custom classes, the expected class can be specified as part of the pattern and is checked with ===

```
class SuperPoint  px)
"matched: #{px}"
else
"not matched"
end
#=> "not matched"
case SuperPoint.new(1, -2)
in SuperPoint[x: 0.. => px] # [] or () parentheses are allowed
"matched: #{px}"
else
"not matched"
end
#=> "matched: 1"
```

These core and library classes implement deconstruction:

* MatchData#deconstruct and MatchData#deconstruct_keys;
* Time#deconstruct_keys, Date#deconstruct_keys, DateTime#deconstruct_keys.

Contoh Kode / Perintah CLI:

```
def deconstruct
puts "deconstruct called"
[@x, @y]
end
def deconstruct_keys(keys)
puts "deconstruct_keys called with #{keys.inspect}"
{x: @x, y: @y}
end
```

[ruby_074] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Guard clauses

Sumber: ht

== Guard clauses
if can be used to attach an additional condition (guard clause) when the pattern matches in case/in expressions.
This condition may use bound variables:
case [1, 2]
in a, b if b == a*2
"matched"
else
"not matched"
end
#=> "matched"
case [1, 1]
in a, b if b == a*2
"matched"
else
"not matched"
end
#=> "not matched"
unless works, too:
case [1, 1]
in a, b unless b == a*2
"matched"
else
"not matched"
end
#=> "matched"
Note that => and in operator can not have a guard clause.
The following examples is parsed as a standalone expression with modifier if.
[1, 2] in a, b if b == a*2

[ruby_075] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Appendix A. Pattern syntax

Sumber: ht

== Appendix A. Pattern syntax
Approximate syntax is:
pattern: value_pattern
| variable_pattern
| alternative_pattern
| as_pattern
| array_pattern
| find_pattern
| hash_pattern
value_pattern: literal
| Constant
| ^local_variable
| ^instance_variable
| ^class_variable
| ^global_variable
| ^(expression)
variable_pattern: variable
alternative_pattern: pattern | pattern | ...
as_pattern: pattern => variable
array_pattern: [pattern, ..., *variable]
| Constant(pattern, ..., *variable)
| Constant[pattern, ..., *variable]
find_pattern: [*variable, pattern, ..., *variable]
| Constant(*variable, pattern, ..., *variable)
| Constant[*variable, pattern, ..., *variable]
hash_pattern: {key: pattern, key:, ..., **variable}
| Constant(key: pattern, key:, ..., **variable)
| Constant[key: pattern, key:, ..., **variable]

[ruby_076] Ruby - Modern Features - Pattern Matching

Pattern Matching (case in, Variable pinning ^, Hash/Array patterns, Alternative patterns |): Appendix B. Some undefined behavior examples

Sumber: ht

== Appendix B. Some undefined behavior examples
To leave room for optimization in the future, the specification contains some undefined behavior.
Use of a variable in an unmatched pattern:
case [0, 1]
in [a, 2]
"not matched"
in b
"matched"
in c
"not matched"
end
a #=> undefined
c #=> undefined

```
Number of deconstruct, deconstruct_keys method calls:
$i = 0
ary = [0]
def ary.deconstruct
$i += 1
self
end
case ary
in [0, 1]
"not matched"
in [0]
"matched"
end
$i #=> undefined
```

[ruby_077] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Overview

Sumber: ht

```
= Modules
Modules serve two purposes in Ruby, namespacing and mix-in functionality.
A namespace can be used to organize code by package or functionality that
separates common names from interference by other packages.  For example, the
IRB namespace provides functionality for irb that prevents a collision
for the common name "Context".
Mix-in functionality allows sharing common methods across multiple classes or
modules.  Ruby comes with the Enumerable mix-in module which provides many
enumeration methods based on the each method and Comparable allows comparison
of objects based on the  comparison method.
Note that there are many similarities between modules and classes.  Besides the
ability to mix-in a module, the description of modules below also applies to
classes.
```

[ruby_078] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Module Definition

Sumber: ht

```
== Module Definition
A module is created using the module keyword:
module MyModule
# ...
end
A module may be reopened any number of times to add, change or remove
functionality:
module MyModule
def my_method
end
end
module MyModule
alias my_alias my_method
end
```

module MyModule
remove_method :my_method
end
Reopening modules (or classes) is a very powerful feature of Ruby, but it is
best to only reopen modules you own. Reopening modules you do not own may lead
to naming conflicts or difficult to diagnose bugs.

[ruby_079] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Nesting

Sumber: ht

== Nesting
Modules may be nested:
module Outer
module Inner
end
end
Many packages create a single outermost module (or class) to provide a
namespace for their functionality.
You may also define inner modules using :: provided the outer
modules (or classes) are already defined:
module Outer::Inner::GrandChild
end
Note that this will raise a NameError if Outer and
Outer::Inner are not already defined.
This style has the benefit of allowing the author to reduce the amount
of indentation. Instead of 3 levels of indentation only one is necessary.
However, the scope of constant lookup is different for creating a namespace
using this syntax instead of the more verbose syntax.

[ruby_080] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): +self+

Sumber: ht

=== self
self refers to the object that defines the current scope. self will change
when entering a different method or when defining a new module.

[ruby_081] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Constants

Sumber: ht

=== Constants
Accessible constants are different depending on the module nesting (which
syntax was used to define the module). In the following example
the constant A::Z is accessible from B as A is part of the
nesting:
module A
Z = 1
module B
p Module.nesting #=> [A::B, A]
p Z #=> 1
end

end
However, if you use :: to define A::B without
nesting it inside A, a NameError exception will be raised because the nesting
does not include A:
module A
Z = 1
end
module A::B
p Module.nesting #=> [A::B]
p Z #=> raises NameError
end
If a constant is defined at the top-level you may preceded it with
:: to reference it:
Z = 0
module A
Z = 1
module B
p ::Z #=> 0
end
end

Contoh Kode / Perintah CLI:

```
module B
p Module.nesting #=> [A::B, A]
p Z #=> 1
end

# ----------------------------------------

module B
p ::Z #=> 0
end
```

[ruby_082] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Methods

Sumber: ht

=== Methods
For method definition documentation see the {syntax documentation for
methods}[rdoc-ref:syntax/methods.rdoc].
Class methods may be called directly. (This is slightly confusing, but a
method on a module is often called a "class method" instead of a "module
method". See also Module#module_function which can convert an instance method
into a class method.)
When a class method references a constant, it uses the same rules as referencing
it outside the method as the scope is the same.
Instance methods defined in a module are only callable when included. These
methods have access to the constants defined when they were included through
the ancestors list:
module A
Z = 1
def z
Z
end

end
include A
p self.class.ancestors #=> [Object, A, Kernel, BasicObject]
p z #=> 1

Contoh Kode / Perintah CLI:

```
def z
z
end
```

[ruby_083] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Visibility

Sumber: ht

=== Visibility
Ruby has three types of visibility. The default is public. A public method
may be called from any other object.
The second visibility is protected. When calling a protected method the
sender must inherit the Class or Module which defines the method. Otherwise a
NoMethodError will be raised.
Protected visibility is most frequently used to define == and
other comparison methods where the author does not wish to expose an object's
state to any caller and would like to restrict it only to inherited classes.
Here is an example:
class A
def n(other)
other.m
end
end
class B 1 -- C is a subclass of B
b.n b #=> 1 -- m called on defining class
a.n b # raises NoMethodError A is not a subclass of B
The third visibility is private. A private method may only be called from
inside the owner class without a receiver, or with a literal self
as a receiver. If a private method is called with a
receiver other than a literal self, a NoMethodError will be raised.
class A
def without
m
end
def with_self
self.m
end
def with_other
A.new.m
end
def with_renamed
copy = self
copy.m
end
def m
1
end

private :m
end
a = A.new
a.without   #=> 1
a.with_self   #=> 1
a.with_other   # NoMethodError (private method `m' called for #)
a.with_renamed # NoMethodError (private method `m' called for #)

Contoh Kode / Perintah CLI:

```
def with_self
self.m
end
def with_other
A.new.m
end
def with_renamed
copy = self
copy.m
end
def m
1
end
private :m
```

[ruby_084] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): +alias+ and +undef+

Sumber: ht

=== alias and undef
You may also alias or undefine methods, but these operations are not
restricted to modules or classes. See the {miscellaneous syntax
section}[rdoc-ref:syntax/miscellaneous.rdoc] for documentation.
= Classes
Every class is also a module, but unlike modules a class may not be mixed-in to
another module (or class). Like a module, a class can be used as a namespace.
A class also inherits methods and constants from its superclass.

[ruby_085] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Defining a class

Sumber: ht

== Defining a class
Use the class keyword to create a class:
class MyClass
...
end
If you do not supply a superclass your new class will inherit from Object. You
may inherit from a different class using followed by a class
name:
class MySubclass < MyClass
...
end
There is a special class BasicObject which is designed as a blank class and

includes a minimum of built-in methods. You can use BasicObject to create an independent inheritance structure. See the BasicObject documentation for further details.
Just like modules, classes can also be reopened. You can omit its superclass when you reopen a class. Specifying a different superclass than the previous definition will raise an error.

```
class C
end
class D < C
end
# OK
class D < C
end
# OK
class D
end
# TypeError: superclass mismatch for class D
class D < String
end
```

[ruby_086] Ruby - Classes, Modules & Data

Classes and Modules (Inheritance, self, include/prepend/extend, Data.define): Inheritance

Sumber: ht

== Inheritance
Any method defined on a class is callable from its subclass:

```
class A
Z = 1
def z
Z
end
end
class B  1
```

The same is true for constants:

```
class A
Z = 1
end
class B  1
```

You can override the functionality of a superclass method by redefining the method:

```
class A
def m
1
end
end
class B  2
```

If you wish to invoke the superclass functionality from a method use super:

```
class A
def m
1
end
```

end
class B 3
When used without any arguments super uses the arguments given to the
subclass method. To send no arguments to the superclass method use
super(). To send specific arguments to the superclass method
provide them manually like super(2).
super may be called as many times as you like in the subclass method.
= Singleton Classes
The singleton class (also known as the metaclass or eigenclass) of an object is
a class that holds methods for only that instance. You can access the
singleton class of an object using class like this:
class C
end
class def self.my_method.
Since you can open the singleton class of any object this means that this code
block:
o = Object.new
def o.my_method
1 1
end
is equivalent to this code block:
o = Object.new
class << o
def my_method
1 1
end
end
Both objects will have a my_method that returns 2.

Contoh Kode / Perintah CLI:

```
def z
z
end
```

[ruby_087] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Overview

Sumber: ht

= Calling Methods
Calling a method sends a message to an object so it can perform some work.
In ruby you send a message to an object like this:
my_method()
Note that the parenthesis are optional:
my_method
Except when there is difference between using and omitting parentheses, this
document uses parenthesis when arguments are present to avoid confusion.
This section only covers calling methods. See also the {syntax documentation
on defining methods}[rdoc-ref:syntax/methods.rdoc].

[ruby_088] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Receiver

Sumber: ht

== Receiver
self is the default receiver. If you don't specify any receiver self will
be used. To specify a receiver use .:
my_object.my_method
This sends the my_method message to my_object. Any object can be a
receiver but depending on the method's visibility sending a message may raise a
NoMethodError.
You may also use :: to designate a receiver, but this is rarely
used due to the potential for confusion with :: for namespaces.

[ruby_089] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Chaining Method Calls

Sumber: ht

=== Chaining Method Calls
You can "chain" method calls by immediately following one method call with another.
This example chains methods Array#append and Array#compact:
a = [:foo, 'bar', 2]
a1 = [:baz, nil, :bam, nil]
a2 = a.append(*a1).compact
a2 # => [:foo, "bar", 2, :baz, :bam]
Details:
- First method merge creates a copy of a,
appends (separately) each element of a1 to the copy, and returns
[:foo, "bar", 2, :baz, nil, :bam, nil]
- Chained method compact creates a copy of that return value,
removes its nil-valued entries, and returns
[:foo, "bar", 2, :baz, :bam]
You can chain methods that are in different classes.
This example chains methods Hash#to_a and Array#reverse:
h = {foo: 0, bar: 1, baz: 2}
h.to_a.reverse # => [[:baz, 2], [:bar, 1], [:foo, 0]]
Details:
- First method Hash#to_a converts a to an \Array, and returns
[[:foo, 0], [:bar, 1], [:baz, 2]]
- Chained method Array#reverse creates copy of that return value,
reverses it, and returns
[[:baz, 2], [:bar, 1], [:foo, 0]]

[ruby_090] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Safe Navigation Operator

Sumber: ht

=== Safe Navigation Operator
&., called "safe navigation operator", allows to skip method call
when receiver is nil. It returns nil and doesn't evaluate method's arguments
if the call is skipped.
REGEX = /(ruby) is (\w+)/i
"Ruby is awesome!".match(REGEX).values_at(1, 2)
=> ["Ruby", "awesome"]
"Python is fascinating!".match(REGEX).values_at(1, 2)
NoMethodError: undefined method `values_at' for nil:NilClass
"Python is fascinating!".match(REGEX)&.values_at(1, 2)
=> nil
This allows to easily chain methods which could return empty value. Note that
&. skips only one next call, so for a longer chain it is necessary
to add operator on each level:
"Python is fascinating!".match(REGEX)&.values_at(1, 2).join(' - ')
NoMethodError: undefined method `join' for nil:NilClass
"Python is fascinating!".match(REGEX)&.values_at(1, 2)&.join(' - ')
=> nil

[ruby_091] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Arguments

Sumber: ht

== Arguments
There are three types of arguments when sending a message, the positional
arguments, keyword (or named) arguments and the block argument. Each message
sent may use one, two or all types of arguments, but the arguments must be
supplied in this order.
All arguments in ruby are passed by reference and are not lazily evaluated.
Each argument is separated by a ,:
my_method(1, '2', :three)
Arguments may be an expression, a hash argument:
'key' => value
or a keyword argument:
key: value
Hash and keyword arguments must be contiguous and must appear after all
positional arguments, but may be mixed:
my_method('a' => 1, b: 2, 'c' => 3)

[ruby_092] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Positional Arguments

Sumber: ht

=== Positional Arguments
The positional arguments for the message follow the method name:
my_method(argument1, argument2)
In many cases, parenthesis are not necessary when sending a message:
my_method argument1, argument2
However, parenthesis are necessary to avoid ambiguity. This will raise a
SyntaxError because ruby does not know which method argument3 should be sent
to:
method_one argument1, method_two argument2, argument3
If the method definition has a *argument extra positional
arguments will be assigned to argument in the method as an Array.
If the method definition doesn't include keyword arguments, the keyword or
hash-type arguments are assigned as a single hash to the last argument:
def my_method(options)
p options
end
my_method('a' => 1, b: 2) # prints: {'a'=>1, :b=>2}
If too many positional arguments are given, an ArgumentError is raised.

[ruby_093] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Default Positional Arguments

Sumber: ht

=== Default Positional Arguments
When the method defines default arguments you do not need to supply all the
arguments to the method. Ruby will fill in the missing arguments in-order.
First we'll cover the simple case where the default arguments appear on the
right. Consider this method:
def my_method(a, b, c = 3, d = 4)
p [a, b, c, d]
end
Here c and d have default values which ruby will apply for you. If you
send only two arguments to this method:
my_method(1, 2)
You will see ruby print [1, 2, 3, 4].
If you send three arguments:
my_method(1, 2, 5)
You will see ruby print [1, 2, 5, 4]
Ruby fills in the missing arguments from left to right.
Ruby allows default values to appear in the middle of positional arguments.
Consider this more complicated method:
def my_method(a, b = 2, c = 3, d)
p [a, b, c, d]
end
Here b and c have default values. If you send only two arguments to this
method:

my_method(1, 4)
You will see ruby print [1, 2, 3, 4].
If you send three arguments:
my_method(1, 5, 6)
You will see ruby print [1, 5, 3, 6].
Describing this in words gets complicated and confusing. I'll describe it
in variables and values instead.
First 1 is assigned to a, then 6 is assigned to
d. This leaves only the arguments with default values. Since
5 has not been assigned to a value yet, it is given to b and
c uses its default value of 3.

[ruby_094] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Keyword Arguments

Sumber: ht

=== Keyword Arguments
Keyword arguments follow any positional arguments and are separated by commas
like positional arguments:
my_method(positional1, keyword1: value1, keyword2: value2)
Any keyword arguments not given will use the default value from the method
definition. If a keyword argument is given that the method did not list,
and the method definition does not accept arbitrary keyword arguments, an
ArgumentError will be raised.
Keyword argument value can be omitted, meaning the value will be fetched
from the context by the name of the key
keyword1 = 'some value'
my_method(positional1, keyword1:)
...is the same as
my_method(positional1, keyword1: keyword1)
Be aware that when method parenthesis are omitted, too, the parsing order might
be unexpected:
my_method positional1, keyword1:
some_other_expression
...is actually parsed as
my_method(positional1, keyword1: some_other_expression)

[ruby_095] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Block Argument

Sumber: ht

=== Block Argument
The block argument sends a closure from the calling scope to the method.
The block argument is always last when sending a message to a method. A block
is sent to a method using do ... end or { ... }:
my_method do
...
end
or:
my_method {

...
}
do end has lower precedence than { } so:
method_1 method_2 {
...
}
Sends the block to method_2 while:
method_1 method_2 do
...
end
Sends the block to method_1. Note that in the first case if parentheses are
used the block is sent to method_1.
A block will accept arguments from the method it was sent to. Arguments are
defined similar to the way a method defines arguments. The block's arguments
go in | ... | following the opening do or
{:
my_method do |argument1, argument2|
...
end
==== Block Local Arguments
You may also declare block-local arguments to a block using ; in
the block arguments list. Assigning to a block-local argument will not
override local arguments outside the block in the caller's scope:
def my_method
yield self
end
place = "world"
my_method do |obj; place|
place = "block"
puts "hello #{obj} this is #{place}"
end
puts "place is: #{place}"
This prints:
hello main this is block
place is: world
So the place variable in the block is not the same place variable as
outside the block. Removing ; place from the block arguments
gives this result:
hello main this is block
place is: block

[ruby_096] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Unpacking Positional Arguments

Sumber: ht

=== Unpacking Positional Arguments
Given the following method:
def my_method(argument1, argument2, argument3)
end
You can turn an Array into an argument list with * (or splat)
operator:

arguments = [1, 2, 3]
my_method(*arguments)
or:
arguments = [2, 3]
my_method(1, *arguments)
Both are equivalent to:
my_method(1, 2, 3)
The * unpacking operator can be applied to any object, not only
arrays. If the object responds to a #to_a method, this method
is called, and is expected to return an Array, and elements of this array are passed
as separate positional arguments:
class Name
def initialize(name)
@name = name
end
def to_a = @name.split(' ')
end
name = Name.new('Jane Doe')
p(*name)
prints separate values:
Jane
Doe
If the object doesn't have a #to_a method, the object itself is passed
as one argument:
class Name
def initialize(name)
@name = name
end
end
name = Name.new('Jane Doe')
p(*name)
Prints the object itself:
#   #
This allows to handle one or many arguments polymorphically. Note also that *nil
is unpacked to an empty list of arguments, so conditional unpacking is possible:
my_method(*(some_arguments if some_condition?))
If #to_a method exists and does not return an Array, it would be an
error on unpacking:
class Name
def initialize(name)
@name = name
end
def to_a = @name
end
name = Name.new('Jane Doe')
p(*name)
can't convert Name to Array (Name#to_a gives String) (TypeError)
You may also use the ** (described next) to convert a Hash into
keyword arguments.
If the number of objects in the Array do not match the number of arguments for
the method, an ArgumentError will be raised.
If the splat operator comes first in the call, parentheses must be used to

avoid an ambiguity of interpretation as an unpacking operator or multiplication operator. In this case, Ru

Contoh Kode / Perintah CLI:

```
def to_a = @name.split(' ')

# ----------------------------------------

def to_a = @name
```

[ruby_097] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Unpacking Keyword Arguments

Sumber: ht

=== Unpacking Keyword Arguments
Given the following method:
def my_method(first: 1, second: 2, third: 3)
end
You can turn a Hash into keyword arguments with the **
(keyword splat) operator:
arguments = { first: 3, second: 4, third: 5 }
my_method(**arguments)
or:
arguments = { first: 3, second: 4 }
my_method(third: 5, **arguments)
Both are equivalent to:
my_method(first: 3, second: 4, third: 5)
The ** unpacking operator can be applied to any object, not only
hashes. If the object responds to a #to_hash method, this method
is called, and is expected to return an Hash, and elements of this hash are passed
as keyword arguments:
class Name
def initialize(name)
@name = name
end
def to_hash = {first: @name.split(' ').first, last: @name.split(' ').last}
end
name = Name.new('Jane Doe')
p(**name)
Prints: {first: "Jane", last: "Doe"}
Unlike * operator, ** raises an error when used on an
object that doesn't respond to #to_hash. The one exception is
nil, which doesn't explicitly define this method, but is still allowed to
be used in ** unpacking, not adding any keyword arguments.
Again, this allows for conditional unpacking:
my_method(some: params, **(some_extra_params if pass_extra_params?))
Like * operator, ** raises an error when the object responds
to #to_hash, but it doesn't return a Hash.
If the method definition uses the keyword splat operator to
gather arbitrary keyword arguments, they will not be gathered
by *:
def my_method(*a, **kw)

p arguments: a, keywords: kw
end
my_method(1, 2, '3' => 4, five: 6)
Prints:
{:arguments=>[1, 2], :keywords=>{'3'=>4, :five=>6}}

Contoh Kode / Perintah CLI:

```
def to_hash = {first: @name.split(' ').first, last: @name.split(' ').last}
```

[ruby_098] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Proc to Block Conversion

Sumber: ht

=== Proc to Block Conversion
Given a method that use a block:
def my_method
yield self
end
You can convert a proc or lambda to a block argument with the &
(block conversion) operator:
argument = proc { |a| puts "#{a.inspect} was yielded" }
my_method(&argument)
If the block conversion operator comes first in the call, parenthesis must be
used to avoid a warning:
my_method &argument # warning
my_method(&argument) # no warning

[ruby_099] Ruby - Calling Methods & Blocks

Calling Methods (Positional/Keyword args, Block arguments &, Safe navigation &.): Method Lookup

Sumber: ht

== Method Lookup
When you send a message, Ruby looks up the method that matches the name of the
message for the receiver. Methods are stored in classes and modules so method
lookup walks these, not the objects themselves.
Here is the order of method lookup for the receiver's class or module R:
* The prepended modules of R in reverse order
* For a matching method in R
* The included modules of R in reverse order
If R is a class with a superclass, this is repeated with R's superclass
until a method is found.
Once a match is found method lookup stops.
If no match is found this repeats from the beginning, but looking for
method_missing. The default method_missing is BasicObject#method_missing
which raises a NameError when invoked.
If refinements (an experimental feature) are active, the method lookup changes.
See the {refinements documentation}[rdoc-ref:syntax/refinements.rdoc] for
details.

[ruby_100] Ruby - Exception Handling

Exceptions (raise, rescue, ensure, else, retry, StandardError): Overview

Sumber: ht

= Exception Handling
Exceptions are rescued in a begin/end block:
begin
code that might raise
rescue
handle exception
end
If you are inside a method, you do not need to use begin or end unless you
wish to limit the scope of rescued exceptions:
def my_method
...
rescue
...
end
The same is true for a class, module, and block:
[0, 1, 2].map do |i|
10 / i
rescue ZeroDivisionError
nil
end
#=> [nil, 10, 5]
You can assign the exception to a local variable by using =>
variable_name at the end of the rescue line:
begin
...
rescue => exception
warn exception.message
raise # re-raise the current exception
end
By default, StandardError and its subclasses are rescued. You can rescue a
specific set of exception classes (and their subclasses) by listing them after
rescue:
begin
...
rescue ArgumentError, NameError
handle ArgumentError or NameError
end
You may rescue different types of exceptions in different ways:
begin
...
rescue ArgumentError
handle ArgumentError
rescue NameError
handle NameError
rescue
handle any StandardError
end
The exception is matched to the rescue section starting at the top, and matches

only once. If an ArgumentError is raised in the begin section, it will not be
handled in the StandardError section.
You may retry rescued exceptions:
begin
...
rescue
do something that may change the result of the begin block
retry
end
Execution will resume at the start of the begin block, so be careful not to
create an infinite loop.
Inside a rescue block is the only valid location for retry, all other uses
will raise a SyntaxError. If you wish to retry a block iteration use redo.
See {Control Expressions}[rdoc-ref:syntax/control_expressions.rdoc] for
details.
To always run some code whether an exception was raised or not, use ensure:
begin
...
r

[ruby_101] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Overview

Sumber: ht

NEWS for Ruby 4.1.0
This document is a list of user-visible feature changes
since the **4.0.0** release, except for bug fixes.
Note that each entry is kept to a minimum, see links for details.

[ruby_102] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Language changes

Sumber: ht

Language changes
* `super` in a module method that was called via `super` from a refinement
method now works correctly.
[[Feature #22309]]
* `Module#clone` and `Module#dup` no longer rewrite the lexical scope of
copied methods. Constants and class variables resolve through the
original class, consistent with inheritance and mixins.
[[Feature #21981]]
* `alias` and `Module#alias_method` in a module emit a deprecation warning
when the original method is not a method of the module or its ancestors
and is only found through the fallback to `Object`. The fallback will be
removed in Ruby 4.3, and such an alias will raise a `NameError`.
[[Bug #22276]]
* `alias` and `Module#alias_method` emit a deprecation warning when the
original method is defined in a prepended module. This behavior will
be removed in Ruby 4.3, and alias lookup will start at the origin
class (after prepended modules).

[[Bug #22273]]

[ruby_103] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Core classes updates

Sumber: ht

Core classes updates
Note: We're only listing outstanding class updates.
* C API
* `RB_NOGVL_PENDING_INTR_FAIL` is added as a flag for `rb_nogvl`.
`rb_nogvl` does not enter the blocking region (and does not call the
given function), returning `0`, if the current thread has pending
interrupts, including interrupts masked by `Thread.handle_interrupt`.
As with the existing skip path, `errno` is `0` because the function was
never called.
* `RHASH_TBL` is deprecated. [[Feature #22232]]
* Array
* `Array#pack` accepts new formats `R` and `r` for unsigned and signed
LEB128 encoded integers. [[Feature #21785]]
* `Array#pack` accepts new formats `x!` and `@!` to align the current
offset to a byte boundary or to the ABI alignment of another
directive. [[Feature #22185]]
* Dir
* `Dir.glob` supports `**/` for recursive matching that follows
directory symbolic links, stopping at links to ancestor directories.
[[Feature #17548]]
* ENV
* `ENV.fetch_values` is added. It returns an array of values for the
given names, raising `KeyError` for missing names unless a block is
given. [[Feature #21781]]
* Fiber::Scheduler
* The `fiber_interrupt` hook is now required. Schedulers which do not
implement it can no longer be installed with `Fiber.set_scheduler`.
* GC
* `GC.stat` accepts a `process:` keyword. `GC.stat` and
`GC.stat(process: false)` report the current Ractor's object space.
With the default collector, `GC.stat(process: true)` reports cumulative
collection counts and measured marking/sweeping CPU time across Ractors,
including history from destroyed object spaces, without requesting a
stop-the-world snapshot. It also reports the `:page_pool_arenas`,
`:page_pool_arenas_freed`, `:page_pool_total_pages` and
`:page_pool_discarded_pages` keys, which the Ractor data does not
include. `

Contoh Kode / Perintah CLI:

```
* `GC.stat` accepts a `process:` keyword. `GC.stat` and
`GC.stat(process: false)` report the current Ractor's object space.
With the default collector, `GC.stat(process: true)` reports cumulative
collection counts and measured marking/sweeping CPU time across Ractors,
including history from destroyed object spaces, without requesting a
stop-the-world snapshot. It also reports the `:page_pool_arenas`,
`:page_pool_arenas_freed`, `:page_pool_total_pages` and
`:page_pool_discarded_pages` keys, which the Ractor data does not
```

```
include. `GC.count` and `GC.total_time` retain their current-Ractor
behavior.
* `GC.stat` reports a new key, `:global_gc_count`: the number of
stop-the-world global garbage collection cycles that collected the
current Ractor's heap, whichever Ractor started them. The new key
`:global_gc_invocation_count` reports the global collections that the
current Ractor started. `GC.stat(:global_gc_count, process: true)`
reports the number of global collections in the process, including
those started by terminated Ractors.
* A global garbage collection is not counted in `:major_gc_count`.
`:count` equals `:minor_gc_count` plus `:major_gc_count` plus
`:global_gc_count` in both the Ractor and the process data. In the
process data each global collection counts once. When another Ractor's
object space exists, including one left by a terminated Ractor that is
not yet absorbed, or one was absorbed since the last global collection,
`GC.start` runs a global collection by default, so it increments
`:global_gc_count` rather than `:major_gc_count`; pass `global: false`
for a local major collection.
* Global compaction no longer counts overlapping CPU intervals in scalar GC
timing statistics. Profiler wall-time intervals are unchanged.

# ----------------------------------------

* `Hash.ruby2_keywords_hash?` and `Hash.ruby2_keywords_hash` are
deprecated and will be removed in Ruby 4.5. [[Feature #22205]]

# ----------------------------------------

* `Integer#bit_count` is added. It returns the number of `1` bits in the
binary representation of a non-negative integer (its population count).
[[Feature #20163]]
```

[ruby_104] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Stdlib updates

Sumber: ht

Stdlib updates
* Psych
* An experimental libfyaml backend has been added. It is only
enabled when psych is built with `--enable-libfyaml`, and the
default libyaml backend remains unchanged otherwise. It is not
supported on Windows. [[GH-psych #805]]
* Ruby::Prism
* `Ruby::Prism` is added. It is the prism parser built into the
interpreter, i.e., the very parser that parsed the running program,
independent of the prism gem. It is autoloaded.
`Method#syntax_tree` and others return its nodes. Its API may change
between Ruby minor versions; use the `prism` gem unless you need the
parser of the running interpreter. [[Bug #22379]]

[ruby_105] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): The following bundled gems are added.

Sumber: ht

The following bundled gems are added.
We only list stdlib changes that are notable feature changes.
Other changes are listed in the following sections. We also listed release
history from the previous bundled version that is Ruby 3.4.0 if it has GitHub
releases.

[ruby_106] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): The following bundled gems are promoted from default gems.

Sumber: ht

The following bundled gems are promoted from default gems.
* prism 1.9.0
* 1.7.0 to [v1.8.0][prism-v1.8.0], [v1.8.1][prism-v1.8.1], [v1.9.0][prism-v1.9.0]
* tsort 0.2.0
* win32-registry 0.1.2

[ruby_107] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): The following bundled gems are removed.

Sumber: ht

The following bundled gems are removed.
* net-ftp 0.3.9
* net-pop 0.1.2
They are still available on rubygems.org and can be installed with
`gem install`. [[Feature #21835]]

[ruby_108] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): The following default gem is added.

Sumber: ht

The following default gem is added.

[ruby_109] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): The following default gems are updated.

Sumber: ht

The following default gems are updated.
* RubyGems 4.1.0.beta2
* 4.0.3 to [v4.0.4][RubyGems-v4.0.4], [v4.0.5][RubyGems-v4.0.5], [v4.0.6][RubyGems-v4.0.6], [v4.0.7][RubyGems-v4.0.7], [v4.0.8][RubyGems-v4.0.8], [v4.0.9][RubyGems-v4.0.9], [v4.0.10][RubyGems-v4.0.10], [v4.0.11][RubyGems-v4.0.11], [v4.0.12][RubyGems-v4.0.12], [v4.0.13][RubyGems-v4.0.13], [v4.0.14][RubyGems-v4.0.14], [v4.0.15][RubyGems-v4.0.15], [v4.0.16][RubyGems-v4.0.16], [v4.0.17][RubyGems-v4.0.17], [v4.0.18][RubyGems-v4.0.18], [v4.0.19][RubyGems-v4.0.19], [v4.0.20][RubyGems-v4.0.20], [v4.0.21][RubyGems-v4.0.21], [v4.0.22][RubyGems-v4.0.22],

[v4.0.23][RubyGems-v4.0.23], [v4.1.0.beta1][RubyGems-v4.1.0.beta1], [v4.1.0.beta2][RubyGems-v4.1.0.beta2]
* bundler 4.1.0.beta2
* 4.0.3 to [v4.0.4][bundler-v4.0.4], [v4.0.5][bundler-v4.0.5], [v4.0.6][bundler-v4.0.6], [v4.0.7][bundler-v4.0.7], [v4.0.8][bundler-v4.0.8], [v4.0.9][bundler-v4.0.9], [v4.0.10][bundler-v4.0.10], [v4.0.11][bundler-v4.0.11], [v4.0.12][bundler-v4.0.12], [v4.0.13][bundler-v4.0.13], [v4.0.14][bundler-v4.0.14], [v4.0.15][bundler-v4.0.15], [v4.0.16][bundler-v4.0.16], [v4.0.17][bundler-v4.0.17]
* date 3.6.0
* 3.5.1 to [v3.6.0][date-v3.6.0]
* delegate 0.7.0
* 0.6.1 to [v0.7.0][delegate-v0.7.0]
* digest 3.3.0
* 3.2.1 to [v3.3.0][digest-v3.3.0]
* erb 6.1.0
* 6.0.1 to [v6.0.1.1][erb-v6.0.1.1], [v6.0.2][erb-v6.0.2], [v6.0.3][erb-v6.0.3], [v6.0.4][erb-v6.0.4], [v6.0.5][erb-v6.0.5], [v6.0.6][erb-v6.0.6], [v6.0.7][erb-v6.0.7]
* error_highlight 0.7.2
* etc 1.5.0
* 1.4.6 to [v1.5.0][etc-v1.5.0]
* fileutils 1.8.1
* 1.8.0 to [v1.8.1][fileutils-v1.8.1]
* io-console 0.9.4
* 0.8.2 to [v0.9.0][io-console-v0.9.0], [v0.9.1][io-console-v0.9.1], [v0.9.2][io-console-v0.9.2], [v0.9.3][io-console-v0.9.3], [v0.9.4][io-console-v0.9.4]
* io-wait 999.999.999
* ipaddr 1.3.0

[ruby_110] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): The following bundled gems are updated.

Sumber: ht

The following bundled gems are updated.
* minitest 6.0.6
* power_assert 3.1.0
* 3.0.1 to [v3.1.0][power_assert-v3.1.0]
* rake 13.4.2
* 13.3.1 to [v13.4.0][rake-v13.4.0], [v13.4.1][rake-v13.4.1], [v13.4.2][rake-v13.4.2]
* test-unit 3.7.8
* 3.7.5 to [3.7.6][test-unit-3.7.6], [3.7.7][test-unit-3.7.7], [3.7.8][test-unit-3.7.8]
* rss 0.3.3
* 0.3.2 to [0.3.3][rss-0.3.3]
* net-imap 0.6.8
* 0.6.2 to [v0.6.3][net-imap-v0.6.3], [v0.6.4][net-imap-v0.6.4], [v0.6.4.1][net-imap-v0.6.4.1], [v0.6.5][net-imap-v0.6.5], [v0.6.6][net-imap-v0.6.6], [v0.6.7][net-imap-v0.6.7], [v0.6.8][net-imap-v0.6.8]
* net-smtp 0.5.2
* 0.5.1 to [v0.5.2][net-smtp-v0.5.2]
* rbs 4.2.0
* 3.10.0 to [v3.10.1][rbs-v3.10.1], [v3.10.2][rbs-v3.10.2], [v3.10.3][rbs-v3.10.3], [v3.10.4][rbs-v3.10.4], [v4.0.0.dev.1][rbs-v4.0.0.dev.1], [v4.0.0.dev.2][rbs-v4.0.0.dev.2], [v4.0.0.dev.3][rbs-v4.0.0.dev.3], [v4.0.0.dev.4][rbs-v4.0.0.dev.4], [v4.0.0.dev.5][rbs-v4.0.0.dev.5], [v4.0.0][rbs-v4.0.0], [v4.0.1.dev.1][rbs-v4.0.1.dev.1], [v4.0.1.dev.2][rbs-v4.0.1.dev.2], [v4.0.1][rbs-v4.0.1], [v4.0.2][rbs-v4.0.2], [v4.0.3][rbs-v4.0.3],

[v4.1.0.pre.1][rbs-v4.1.0.pre.1], [v4.1.0.pre.2][rbs-v4.1.0.pre.2], [v4.1.0][rbs-v4.1.0], [v4.1.1.pre.1][rbs-v4.1.1.pre.1], [v4.1.1][rbs-v4.1.1], [v4.1.2][rbs-v4.1.2], [v4.1.3][rbs-v4.1.3], [v4.2.0.pre.1][rbs-v4.2.0.pre.1], [v4.2.0][rbs-v4.2.0]
* typeprof 0.33.2
* mutex_m 0.3.0
* bigdecimal 4.1.3
* 4.0.1 to [v4.1.0][bigdecimal-v4.1.0], [v4.1.1][bigdecimal-v4.1.1], [v4.1.2][bigdecimal-v4.1.2], [v4.1.3][bigdecimal-v4.1.3]
* resolv-replace 0.2.0
* 0.1.1 to [v0.2.0][resolv-replace-v0.2.0]
* nkf 0.3.0
* 0.2.0 to [v0.3.0][nkf-v0.3.0]
* syslog 0.4.0
* 0.3.0 to [v0.4.0][syslog-v0.4.0]
* csv 3.3.6
* 3.3.5 to [v3.3.6][csv-v3.3.6]
* repl_type_completor 0.1.16
* 0.1.12 to [v0.1.13][repl_type_completor-v0.1.13],

[ruby_111] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): RubyGems and Bundler

Sumber: ht

RubyGems and Bundler
Ruby 4.0 bundled RubyGems and Bundler version 4. see the following links for details.

[ruby_112] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Supported platforms

Sumber: ht

Supported platforms
* Support code for the following platforms has been removed. Their vendors or distributors have ended support, there is no environment to test Ruby on them, and none of them has a platform maintainer.
* AmigaOS
* AtheOS
* BSD/OS (BSDi)
* ESIX
* EWS-UX (NEC EWS4800)
* FreeBSD, NetBSD and OpenBSD with a.out binaries
* GNU/kFreeBSD
* GNU/kOpenSolaris
* HI-UX/MPP
* HP-UX on the HP 9000 Series 300
* Interix (Windows Services for UNIX)
* IRIX
* MirOS BSD
* NeXTSTEP, OpenStep and Rhapsody
* OSF/1 (Tru64 UNIX)
* QNX Neutrino
* SunOS 4 (Solaris, i.e. SunOS 5, is unaffected)
* System V Release 4

* UNICOS/mp (Cray X1)
* UX/PDS
* Support code for the following old versions has been removed.
* Cygwin 1.3 and earlier
* Solaris 9 and earlier
* Linux 2.6.23 and earlier (RHEL 5)
* Windows 10 1703 or later no longer needs the `LongPathsEnabled` registry value to use paths longer than 260 characters. This applies to any process running the interpreter, including a program which embeds libruby. Each path component is still limited to 255 characters, and a child process still starts with the `MAX_PATH` limited current directory. [[Bug #18947]]
* On 64-bit Windows with MSVC (mswin), String, Array and MatchData can now be 2GiB or longer. Their lengths used to be `long`, which is 32 bits there. [[Feature #22400]]
* Building Ruby with MSVC now requires Visual Studio 2017 version 15.8 (`_MSC_VER` 1915) or later.
* Ruby built with MSVC now requires Windows 10 version 1809 (build 17763) or Windows Server 2019 or later. Earlier Windows 10 releases and Windows Server 2016 are no longer supported.
* Ruby on macOS now requires Mac OS X 10.6 or later. Mac OS X 10.5 and PowerPC Macs are no longer supported.

Contoh Kode / Perintah CLI:

```
* Cygwin 1.3 and earlier
* Solaris 9 and earlier
* Linux 2.6.23 and earlier (RHEL 5)
```

[ruby_113] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Compatibility issues

Sumber: ht

Compatibility issues
* A class or module can now be modified only by the Ractor which created it, its *owner*. Defining, removing or undefining methods, `alias`, changing visibility, `include`/`prepend`, `Module#refine`, defining or removing constants, registering an `autoload`, writing the class's own instance variables and class variables, `Module#freeze` and `Module#set_temporary_name` raise `Ractor::IsolationError` in any other Ractor. Reading is unchanged. Everything defined at boot or by the main Ractor, `require`d libraries included, is owned by the main Ractor, so a non-main Ractor can no longer monkey-patch it; and since defining a constant in a foreign class is prohibited, it can not define a top-level class or module name either. In exchange a Ractor has full use of the classes it creates itself, including unshareable constant, instance variable and class variable values, which not even the main Ractor could do before. See doc/language/ractor.md. [[Feature #22226]]
* `Kernel#at_exit` and `END {}` now raise `Ractor::IsolationError` when called in a non-main Ractor. Previously the registered handler ran in the main Ractor at process exit, which was confusing. [[Feature #22139]]
* `Ractor.make_shareable` now raises `Ractor::Error` for an `IO` object, and `Ractor.shareable?` returns `false` for it. A frozen `IO` used to be shareable, but almost all of its methods raise `FrozenError` and it can

still refer to unshareable objects through the members of `rb_io_t`.
* `Random.urandom` now raises `RangeError` for every size too large for a
String, as `Random#bytes` does. A size that still fit in an `unsigned long`
used to raise `ArgumentError`. [[Feature #22400]]
* `Struct#[]` and `Struct#[]=` now raise `IndexError` for every out-of-range
Integer index withi

[ruby_114] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Stdlib compatibility issues

Sumber: ht

Stdlib compatibility issues
* prism is now a bundled gem. Under Bundler, `require "prism"` needs prism
in the Gemfile. Use `Ruby::Prism` for the parser of the running
interpreter. [[Bug #22379]]
* `Etc.getlogin` on Windows now returns the login name determined when the
process starts, from the `USER` or `USERNAME` environment variable or
`GetUserName()`. It used to follow later changes to `ENV['USER']`.
* Socket
* On Windows, a connection that is refused or unreachable now raises the
matching `Errno` class as soon as Winsock reports it, instead of
`Errno::ETIMEDOUT` once the whole `connect_timeout` has passed. Code
rescuing `Errno::ETIMEDOUT` there has to rescue the real error instead.
* On Windows, `BasicSocket#getsockopt(:SOCKET, :ERROR)` now reports an
errno as it does on the other platforms, instead of the raw WinSock
error code. Code comparing it with a `WSAE*` value has to compare it
with the matching `Errno::*::Errno` instead.
[[Bug #18661]]

Contoh Kode / Perintah CLI:

```
* On Windows, a connection that is refused or unreachable now raises the
matching `Errno` class as soon as Winsock reports it, instead of
`Errno::ETIMEDOUT` once the whole `connect_timeout` has passed. Code
rescuing `Errno::ETIMEDOUT` there has to rescue the real error instead.
* On Windows, `BasicSocket#getsockopt(:SOCKET, :ERROR)` now reports an
errno as it does on the other platforms, instead of the raw WinSock
error code. Code comparing it with a `WSAE*` value has to compare it
with the matching `Errno::*::Errno` instead.
```

[ruby_115] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Embedded TypedData

Sumber: ht

Embedded TypedData
* The `RUBY_TYPED_EMBEDDABLE` flag is now public and documented and can be used by C extensions.
It allows allocating C structs directly into Ruby object slots, which reduces pointer chasing,
and in some case memory usage.
See the C extension documentation for details. [[Feature #21853]]
* Added new C23 inspired allocator functions, that takes the previous memory size.
This allow the Ruby GC to better keep track of memory usage, improving its heuristics.
It also improves the performance of system allocators that support C23 `free_sized`.
However, it is important to note that passing an incorrect size to these function is undefined

behavior and may result in crashes or memory leaks.
- `ruby_xfree_sized(void *ptr, size_t size)`
- `ruby_xrealloc_sized(void *ptr, size_t newsiz, size_t oldsiz)`
- `ruby_xrealloc2_sized(void *ptr, size_t newelems, size_t newsiz, size_t oldelems)`
[[Feature #21861]]

[ruby_116] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Ractor-scoped GC address registration

Sumber: ht

Ractor-scoped GC address registration
* `rb_gc_register_address` and `rb_global_variable` now register the address with the calling Ractor, and only that Ractor's GC marks the stored object. Any Ractor may register an address, but the value stored through it must be a special constant, a shareable object, or an unshareable object owned by the registering Ractor. Storing another Ractor's unshareable object can result in the object being freed while the address still refers to it (use-after-free). If the address has process lifetime (a static VALUE), register it from the main Ractor or keep the stored values shareable. When the registering Ractor is joined with `Ractor#value`, remaining registrations move to the joining Ractor; otherwise they move to the main Ractor once the dead Ractor is collected.
[[Feature #22277]]

[ruby_117] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Type for lengths and indices

Sumber: ht

Type for lengths and indices
* `rb_len_t` and `rb_ulen_t` are added as the types of the lengths and indices of String, Array and MatchData, with `LEN2NUM`, `NUM2LEN`, `ULEN2NUM`, `rb_len2int`, `PRIdLEN` and its siblings, `RB_LEN_MAX`, `RB_LEN_MIN`, `RB_ULEN_MAX`, `SIZEOF_RB_LEN_T` and `HAVE_RB_LEN_T`. `RbConfig::SIZEOF["rb_len_t"]` gives its size to an extconf.rb. They are `long`, except on 64-bit mswin, where they are 64 bits. The C API that carries such lengths, such as `RSTRING_LEN` and `rb_range_beg_len`, uses them.
[[Feature #22400]]
* On 64-bit mswin this changes the ABI, so extensions have to be rebuilt. An extension that passes a `long *` to an output parameter such as `rb_range_beg_len` no longer compiles there, and one that keeps a length in `long` truncates it at 2GiB. Keep such lengths in `rb_len_t`, or convert them with `rb_len2long`. An extension that also supports older versions can check `HAVE_RB_LEN_T` and fall back to `typedef long rb_len_t`.
[[Feature #22400]]
* On mswin, extensions are now built with incompatible pointer types (warning C4133) as errors. Besides a `long *` passed where an `rb_len_t *` is expected, this rejects any other mismatched pointer that used to compile with a warning.
[[Feature #22400]]

* `rb_len2long` is added to convert an `rb_len_t` to `long`, raising
`RangeError` via the new `rb_out_of_long` when it does not fit, for
extensions that keep a length in `long`. [[Feature #22400]]

[ruby_118] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Atomic operations

Sumber: ht

Atomic operations
* `RUBY_ATOMIC_SIZE_LOAD` is added as the `size_t` counterpart of
`RUBY_ATOMIC_LOAD`, alongside the existing `RUBY_ATOMIC_SIZE_ADD`,
`RUBY_ATOMIC_SIZE_SUB`, `RUBY_ATOMIC_SIZE_CAS` and `RUBY_ATOMIC_SIZE_EXCHANGE`.

[ruby_119] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Removed APIs

Sumber: ht

Removed APIs
The following APIs, which have been deprecated for many years, are removed.
[[Feature #21768]]
* old postponed job functions,
* untyped data object type/functions,
* old APIs to allocate a data object,
* taintedness/trustedness enums/macros,
* `rb_gc_force_recycle` function,
* `rb_iterate` function,
* and some functions and constants for internal use.

[ruby_120] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Implementation improvements

Sumber: ht

Implementation improvements
- The default garbage collector has been switched from a freelist allocator to a bump pointer allocator. [[PR #17201]]
- The error_highlight, did_you_mean, and syntax_suggest gems are now
loaded lazily on the first error display instead of at interpreter
boot, which reduces startup time. `Process.warmup` loads them
eagerly so that pre-forking servers keep their code in
copy-on-write shared memory. [[Feature #21951]]

[ruby_121] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): Ractor

Sumber: ht

Ractor
A lot of work has gone into making Ractors more stable, performant, and usable. These improvements bring Ractor implementation closer to leaving experimental status.
* The default GC now runs **per Ractor**: each Ractor collects its own heap
on its own thread without stopping the others, and a stop-the-world

collection only runs when it is really needed (explicit full `GC.start`,
shareable-object growth, reclaiming dead Ractors' heaps). Allocation-heavy
Ractor programs now scale like forked processes.
* `GC.start(global: false)` can be used when multiple Ractors are running to force
a Ractor-local GC. By default, `GC.start` runs a global GC (all Ractors) like
before. This can also be triggered with `GC.start(global: true)`. The `global`
keyword argument has no effect when a single Ractor is running.
Visible behavior changes:
* `Ractor#value` returns the value only once; a second call raises
`Ractor::Error`.
* `GC.disable`/`GC.enable` act as per-Ractor holds on a process-wide
switch: one Ractor's `GC.enable` no longer overrides another Ractor's
`GC.disable`.
* `ObjectSpace.each_object` enumerates the calling Ractor's own objects
plus other Ractors' shareable objects (`ObjectSpace.dump_all` still
covers everything).
* `ObjectSpace.define_finalizer` on another Ractor's object raises
`Ractor::IsolationError`.
* `Ractor#monitor` now sends an Array naming the Ractor and what happened to
it, `[ractor, :exited]` or `[ractor, :aborted]`, where it used to send the
bare Symbol `:exited` or `:aborted`. Several Ractors can then report to one
port and the receiver still knows which one finished. The Array is built for
the receiving Ractor, so watching many Ractors leaves no shareable objects
behind.
r = Ractor.new { :ok }
r.monitor(port = Ractor::Port.new)
port.receive

Contoh Kode / Perintah CLI:

```
r = Ractor.new { :ok }
r.monitor(port = Ractor::Port.new)
port.receive #=> [r, :exited]
```

[ruby_122] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): M:N thread scheduler

Sumber: ht

M:N thread scheduler
* The scheduler scales with the number of waiters and of Ractors, where it
used to walk a list or take one lock for all of them:
* A timed wait sits in a hierarchical timer wheel rather than on a list
sorted by deadline, which was inserted into by a linear scan.
* An fd stays armed in the backend between waits, instead of being added
before each wait and removed after each wake.
* The io-wait bookkeeping is sharded by fd, rather than serialized on one
lock across every fd.
* A timed wait on an fd rides the scheduler instead of going to a blocking
region, which cost a native thread handoff per wait.
* A context switch, and leaving or rejoining the shared pool, no longer take
the scheduler's global lock.
* The `RUBY_MN_THREADS` environment variable now runs from no M:N scheduling
at all to all of it. `-1` is new: a Ractor's threads have been M:N since

the scheduler was added, with no way to turn that off. `0` and `1` are
unchanged.
	main thread	the main Ractor's other threads	a Ractor's threads
`-1`	1:1	1:1	1:1
`0` or unset	1:1	1:1	M:N
`1`	1:1	M:N	M:N
`2`	M:N	M:N	M:N
* `RUBY_MN_THREADS=2` is new. The main thread is resumed like any other M:N
thread rather than woken on a native thread of its own, which costs an order
of magnitude more. It pays off when the main thread drives the work, and
does nothing for one that only starts other threads and waits.
The main thread is then no longer bound to one OS thread, which is what the
M:N scheduler already meant for every other thread:
* A C extension that keeps state per OS thread has to call
`rb_thread_lock_native_thread()`.
* What must run on the process's initial thread does not work at all,
pinning included: macOS AppKit and CFRunLo

[ruby_123] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): ZJIT

Sumber: ht

ZJIT
* Status
* ZJIT is now faster than YJIT on many benchmarks, but we expect that
ZJIT can be still slower than YJIT on larger benchmarks or applications
as of Ruby 4.1.0 preview releases. Stay tuned for the Ruby 4.1.0 release.
* Major optimizations
* Method inlining: small Ruby methods, including methods that yield to
a block, are inlined into their callers, so that the optimizer can see
across method boundaries.
* Lightweight frames: method calls in JIT code no longer write a full
VM control frame. Frame metadata used by backtraces and GC is
materialized only when needed.
* Optimization pass: a new linear scan register allocator, load-store
optimization, more constant folding, and redundant guard elimination.
* GC fastpath: allocation of objects is inlined into JIT code, skipping
a call into the GC on the fast path.
* More Ruby code is compiled
* Calls with blocks or `&block` arguments, `super`, splats, keyword
arguments, `...` forwarding, and post-required parameters.
* Specialized code for many core methods, e.g. `Float` arithmetic,
`Array#[]`, `Array#length`, `Array#empty?`, `Integer#[]`,
`String#byteslice`, `String#to_sym`, `Class#allocate`, and `new` on
`Struct` classes.
* Instance variable reads and shareable constants are now compiled
while multiple Ractors are running.
* Command-line options
* `--zjit-stats=FILE` writes stats as JSON when `FILE` ends with `.json`.
* Ruby API

* `RubyVM::ZJIT.exit_locations` and `RubyVM::ZJIT.dump_exit_locations`
are removed along with the Stackprof output of `--zjit-trace-exits`.

[ruby_124] Ruby - Modern Features - Ruby 3.2 / 3.3

Ruby 3.2 & 3.3 Modern Release Features (YJIT Performance, Data class, Prism Parser): YJIT

Sumber: ht

YJIT
* YJIT now supports Windows (mswin) on x64 CPUs. Configure with
`win32\configure.bat --enable-yjit`, which requires `rustc` with the
`x86_64-pc-windows-msvc` target. Only the release build is supported,
and MinGW is not. [[Feature #18439]]
* Add `RubyVM::YJIT.max_compile_time_ns` and `RubyVM::YJIT.max_compile_time_ns=`
to give YJIT a compilation time budget. When `RubyVM::YJIT.total_compile_time_ns`
exceeds it, YJIT pauses compilation until the budget is raised, which
spreads the warmup overhead over a longer period. [[Feature #22236]]
[Bug #18661]: https://bugs.ruby-lang.org/issues/18661
[Bug #18947]: https://bugs.ruby-lang.org/issues/18947
[Bug #22273]: https://bugs.ruby-lang.org/issues/22273
[Bug #22276]: https://bugs.ruby-lang.org/issues/22276
[Bug #22379]: https://bugs.ruby-lang.org/issues/22379
[Feature #8948]: https://bugs.ruby-lang.org/issues/8948
[Feature #9779]: https://bugs.ruby-lang.org/issues/9779
[Feature #15330]: https://bugs.ruby-lang.org/issues/15330
[Feature #17548]: https://bugs.ruby-lang.org/issues/17548
[Feature #18439]: https://bugs.ruby-lang.org/issues/18439
[Feature #20163]: https://bugs.ruby-lang.org/issues/20163
[Feature #21390]: https://bugs.ruby-lang.org/issues/21390
[Feature #21768]: https://bugs.ruby-lang.org/issues/21768
[Feature #21781]: https://bugs.ruby-lang.org/issues/21781
[Feature #21785]: https://bugs.ruby-lang.org/issues/21785
[Feature #21796]: https://bugs.ruby-lang.org/issues/21796
[Feature #21835]: https://bugs.ruby-lang.org/issues/21835
[Feature #21853]: https://bugs.ruby-lang.org/issues/21853
[Feature #21861]: https://bugs.ruby-lang.org/issues/21861
[Feature #21932]: https://bugs.ruby-lang.org/issues/21932
[Feature #21951]: https://bugs.ruby-lang.org/issues/21951
[Feature #21981]: https://bugs.ruby-lang.org/issues/